

INTERNAL ENGINEERING DOCUMENTATION

Orbyte

Technical Architecture & Onboarding Guide — Frontend,
Backend, and the AI/Data Processing Pipeline, explained end
to end from the actual codebase.

Next.js · React · TypeScript

FastAPI · SQLAlchemy · Celery

OpenSearch · Postgres · Redis

LiteLLM · Ollama-compatible

Prepared for engineers joining the Orbyte project

Contents

1. Tech Stack
2. High-Level Architecture
 - App Router usage
 - ``proxy.ts`` — Next.js Middleware
 - How the frontend talks to the backend
3. Directory Structure (``web/src/``)
4. State Management & Data Fetching
 - SWR is the fetching layer; ``lib/swr-keys.ts`` is the key registry
 - The ``svc.ts`` + ``hooks.ts`` pairing
 - Global Context providers
5. API Communication Layer
 - The ``svc.ts`` convention, in full, using Groups as the reference example
 - Error handling primitives (``src/lib/fetcher.ts``)
 - Auth / session handling: cookies, not tokens, and the browser never touches the token
6. Component Library / Design System
 - Opal (``@onyx-ai/opal``, ``web/lib/opal/src/``)
 - Theming / color tokens
7. Major Feature Areas
 - 7.1 Chat interface (``/app``)
 - 7.2 Agents / Personas
 - 7.3 Admin panel overview
 - 7.4 Groups & Curators (NEW — Orbyte-specific)
 - 7.5 Join Links (NEW — Orbyte-specific)
 - 7.6 Connectors (brief pass)
 - 7.7 Craft (code execution / build UI) — brief pass
 - 7.8 Query History (brief pass)
 - 7.9 Auth pages
8. Existing Frontend "APIs" Reference (key ``svc.ts``/hook files by feature)
9. Testing
1. Tech Stack
 - Core web/API
 - Database / persistence
 - Search / vector index
 - Object storage
 - Background processing
 - LLM layer
 - Observability / tracing
 - Connectors / document processing
 - Craft (code-execution) sandbox
 - Packaging / tooling
2. High-Level Architecture
 - Request lifecycle (simple chat example)

Request lifecycle (document indexing, asynchronous)

Model server

3. Directory Structure

`backend/onyx/` (Community edition — the base of the product)

`backend/ee/onyx/` (Enterprise-tier code that remains)

4. Database Layer Deep Dive

Core entities (all in `onyx/db/models.py`, one shared `Base` / one Postgres schema per tenant)

Alembic migrations

5. Auth & Permissions Deep Dive

fastapi-users setup

RBAC / `UserRole` and `Permission`

Groups / Curators (NEW to this deployment)

Join Links (NEW to this deployment — full flow)

6. RAG / Search / Indexing Pipeline — Deep Algorithmic Dive

End-to-end pipeline and the real Celery task/worker names

Chunking strategy

Embedding model

Reranking

Hybrid search algorithm (OpenSearch backend — `onyx/document\_index/opensearch/search.py`)

Document permissions/ACL propagation into the index

Citation processing (grounding answers back to source chunks)

7. Chat / Agent Orchestration Deep Dive

Persona/agent config resolution

Prompt construction and the LLM abstraction

Tool-calling loop

Streaming response mechanism

Deep Research (multi-step agentic flow — still present)

8. Craft (Code Execution / Sandbox) Deep Dive

What it is

Sandbox architecture — two selectable backends (`SANDBOX\_BACKEND` env var)

Session / turn lifecycle

9. Background Workers (Celery) Deep Dive

Beat schedule (`onyx/background/celery/tasks/beat\_schedule.py::get\_tasks\_to\_schedule`)

The no-Celery deployment mode

10. API Layer / Server Routers

Error-handling convention

11. Existing APIs Reference

Auth (fastapi-users, Basic only)

Users & permissions

Groups / Curators / Join Links (new)

Connectors, credentials, cc-pairs

Personas / Agents / Document Sets / Hierarchy / Projects / Tools / Skills

Chat & Search

Web search & content providers

LLM / Embedding / Search-settings admin

Admin/settings/security/tracing

Bots

Craft (`/build/\*`, `/sessions/\*`, `/approvals/\*`, `/user-library/\*`, `/scheduled-tasks/\*`)

Federated search & Knowledge Graph

EE-only: Analytics, Query History, Reporting, License, Evals, Standard Answers

12. Observability / Tracing

The `LLMFlow` tagging convention



PART ONE

Frontend Architecture

Next.js App Router structure, state management, the component & design system, the API communication layer, and every major feature area — including the two features built specifically for this product: Groups & Curators, and Join Links.

Orbyte is a rebranded, trimmed-down fork of Onyx. Its frontend lives entirely under `web/` in this repo. This document is grounded in the actual source at `C:\Users\ELITE\Desktop\onyx-main\onyx-main\web` (read directly, not inferred) as of this writing.

1. Tech Stack

Source: `web/package.json`.

Package	Version	Role
<code>next</code>	16.2.10	App Router framework — routing, SSR/RSC, the dev-mode API proxy, middleware (<code>proxy.ts</code>).
<code>react</code> / <code>react-dom</code>	19.2.4	UI runtime. React Compiler (<code>babel-plugin-react-compiler</code>) is enabled for production builds only — disabled in <code>next dev</code> because it roughly doubles dev-server memory under Turbopack (see comment in <code>next.config.js</code>).
<code>typescript</code>	<code>^5.9.3</code> (build), <code>typescript-7</code> (type-checking script)	Static typing. <code>types:check</code> runs <code>next typegen</code> then a separate TS7 compiler in no-emit mode against <code>tsconfig.types.json</code> — a two-stage type-check, not the normal <code>tsc</code> .
<code>tailwindcss</code>	<code>^4.3.0</code> (<code>@tailwindcss/postcss</code>)	Utility CSS. Config lives in <code>web/tailwind-themes/tailwind.config.js</code> , which layers on top of a shared Opal preset (<code>@onyx-ai/opal/tailwind-preset</code>).

Package	Version	Role
<code>@onyx-ai/opal</code> (<code>web/lib/opal</code>)	workspace package (<code>file:./lib/opal</code>)	Orbyte's own component library / design system ("Opal"). Not an external dependency — it's a local workspace package built with <code>tsup</code> , versioned separately, and consumed via subpath exports (<code>@opal/components</code> , <code>@opal/layouts</code> , <code>@opal/icons</code> , <code>@opal/core</code> , etc. — see §6).
<code>@onyx-ai/shared</code> (<code>web/lib/shared</code>)	workspace package	Design tokens (colors, spacing, typography) as JSON, compiled by Style Dictionary into <code>tokens.css</code> . Shared between the web app and Opal so both consume the same token source.
<code>swr</code>	^2.1.5	Client-side data fetching/caching. This is the <i>only</i> data-fetching library used app-wide — there is no React Query, no Apollo, no RTK Query.
<code>formik</code> + <code>yup</code>	^2.2.9 / ^1.4.0	Form state management and schema validation, respectively. Used for every non-trivial form (login, signup, join-link redemption, agent editor, group editor inputs where relevant).
<code>zustand</code>	^5.0.8	Lightweight non-SWR client state for a few hot paths that need imperative, non-React-render-cycle state — e.g. the chat session store (<code>src/app/app/stores/useChatSessionStore.ts</code>).
<code>@dnd-kit/*</code>	^6-8.x	Drag-and-drop (e.g. reordering agents, sidebar items).
<code>@radix-ui/*</code>	various	Unstyled, accessible primitives (dialog, popover, dropdown, tabs, accordion, etc.) that Opal's higher-level components build on top of.
<code>@sentry/nextjs</code>	^10.54.0	Error monitoring; wraps <code>next.config.js</code> via <code>withSentryConfig</code> . Only active when <code>SENTRY_AUTH_TOKEN</code> + <code>NEXT_PUBLIC_SENTRY_DSN</code> are set.
<code>posthog-js</code>	^1.376.3	Product analytics; proxied same-origin through <code>/ph_ingest/*</code> rewrites in <code>next.config.js</code> (avoids ad-blockers and keeps CSP <code>connect-src</code> narrow).
<code>js-cookie</code>	^3.0.5	Reading/writing non-HttpOnly cookies client-side (e.g. sidebar folded state). The actual auth cookie is HttpOnly and never touched by JS — see §5.
<code>react-markdown</code> + <code>remark-*</code> / <code>rehype-*</code> + <code>katex</code> / <code>highlight.js</code>	various	Chat message rendering pipeline: Markdown → sanitized HTML, GFM tables, math (KaTeX), and code syntax highlighting.
<code>jest</code> + <code>@testing-library/react</code>	^29 / ^16	Unit / component tests (<code>*.test.ts(x)</code> co-located with source).
<code>@playwright/test</code>	^1.39	End-to-end tests, in <code>web/tests/e2e</code> (see §9).
<code>storybook</code>	^8.6.18	Component-level visual development, mostly for <code>refresh-components</code> /Opal-adjacent components (<code>.stories.tsx</code> files sit next to components, e.g. <code>src/sections/input/BaseInputBar.stories.tsx</code>).

Package manager & tooling — `package.json` scripts confirm:

- bun** is the intended package manager/runner (`bun run clean`, `bunx playwright test` per `AGENTS.md`), and workspaces (`lib/opal`, `lib/shared`) are resolved via `bun.lock`. `next.config.js` explicitly pins `turbopack.root` to the `web/` directory specifically because Next.js gets confused by the two `bun.lock` files (repo root + `web/`) and picks the wrong workspace root otherwise.
- oxlint** (`oxlint`) and **oxfmt** (`oxfmt`) are the linter/formatter — not ESLint/Prettier. `npm run lint` → `oxlint`, `npm run format` → `oxfmt src`. These are Rust-based, much faster alternatives that this codebase has standardized on.

2. High-Level Architecture

App Router usage

Routing is 100% Next.js App Router under `web/src/app/`. Every folder is a route segment; `page.tsx` is the leaf render, `layout.tsx` wraps it and any siblings. Route groups aren't heavily used — the tree is mostly literal (`/admin/groups/[id]`, `/app/agents/create`, `/join/[token]`).

Server vs Client Components — the convention in this codebase is:

- `layout.tsx` files are **async Server Components** that do the one thing RSC does well here: *authentication gating before any client JS ships*. E.g. `src/app/app/layout.tsx` calls `await requireAuth()` and redirects server-side if unauthenticated, *before rendering* `<AppSidebar>` / `<AppChrome>`. `src/app/admin/layout.tsx` similarly delegates to `AdminSSChrome` → `requireAdminAuth()`.
- Everything below that — essentially all interactive UI — is **"use client"**. Actual data fetching is explicitly **not** done server-side; the `AppProvider` doc comment states this directly: "All data is fetched client-side by individual providers via SWR hooks, eliminating server-side data fetching from the root layout and preventing RSC prefetch amplification." This is a deliberate choice, not an oversight — the team explicitly avoided fetching data in Server Components to keep RSC payloads small and avoid double-fetching.
- A handful of pages read data server-side for redirect decisions only (e.g. `src/app/join/[token]/page.tsx` calls `getCurrentUserSS()` purely to decide whether to redirect an already-logged-in user to `/app` — the actual join-link data is then fetched client-side via SWR inside `JoinLinkForm`).

`proxy.ts` — Next.js Middleware

`web/src/proxy.ts` is the middleware entry point (Next's `proxy` export, matched against almost every route via `config.matcher`). It does three unrelated jobs in one file:

- CSP headers** — builds and attaches a `Content-Security-Policy` header to every response. This is done in middleware rather than baked into `next.config.js` `headers()` specifically because two of the flags (`WEB_FRAME_PROTECTION_ENABLED`, `WEB_STRICT_CSP_ENABLED`) are read from `process.env` **at request time**, so ops can flip them on a running deployment without a rebuild — a config-in-build-artifact would require a redeploy.
- Edge auth fast-fail** — `PROTECTED_ROUTES = ["/admin", "/agents", "/connector"]` get an early redirect to `/auth/login` if there's no auth cookie at all. This is explicitly a *defense-in-depth, cheap* check — the code comment says the real authorization (role checks, token validity) happens in each layout via `requireAuth()` / `requireAdminAuth()`. Notably, `/app` (the chat UI) is **excluded** from this edge check: anonymous-user access to `/app` is gated by a runtime, Redis-backed setting the edge middleware can't read from a cookie, so that check is pushed down to the `requireAuth()` call in `src/app/app/layout.tsx` instead.
- EE route rewriting** — `EE_ROUTES` is a short, explicit allowlist: `/admin/performance/usage`, `/admin/performance/query-history`, `/admin/standard-answer`, `/agents/stats`. When `SERVER_SIDE_ONLY_PAID_ENTERPRISE_FEATURES_ENABLED` is true, requests to these paths are `NextResponse.rewrite()` -n to the same path under `/ee/*` (e.g. `/admin/performance/query-history` → `/ee/admin/performance/query-history`), which is a *separate, parallel page tree* under `src/app/ee/`. This lets Community-edition builds ship without the EE code paths ever being routed to, while Enterprise builds transparently swap in the EE version of the same URL. **Groups is explicitly excluded from this list** — a comment in `proxy.ts` notes Groups is now a first-party Orbyte feature with its own MIT-tier backend (`/manage/admin/groups`), unlike upstream Onyx where group management was EE-gated.

How the frontend talks to the backend

There is **no bespoke backend-facing REST client** — the app calls the FastAPI backend through plain `fetch()` calls to same-origin `/api/*` paths (e.g. `fetch("/api/me")`, `fetch(SWR_KEYS.adminUserGroups)`). Cookies are attached automatically (or explicitly via `credentials: "include"` on cross-cutting calls like `basicLogin`).

Whether that `/api/*` path physically proxies to FastAPI depends on environment:

- Development:** `web/src/app/api/[...path]/route.ts` is a catch-all Next.js Route Handler (`GET` / `POST` / `PUT` / `PATCH` / `DELETE` / `HEAD` / `OPTIONS` all delegate to one `handleRequest`). It forwards the request (method, headers, body, including streamed bodies for chat SSE) to `${INTERNAL_URL}/${path}` (the FastAPI backend, default `http://localhost:8080`), copies back `set-cookie` headers (Next's `fetch` normally drops multiple `Set-Cookie` headers, hence the manual `getSetCookie()` / `append` dance), and preserves chunked/streaming responses (adds `Cache-Control: no-cache, no-transform` and `X-Accel-Buffering: no` when it detects a stream).
- Production:** the same route handler **actively refuses to run** — `if (process.env.NODE_ENV !== "development" && OVERRIDE_API_PRODUCTION !== "true") return 404`. The comment is explicit: "In production, something else (e.g. nginx) should handle this." So in a real deployment, `/api/*` is expected to be routed straight to the FastAPI backend by a reverse proxy in front of both services — the Next.js app never sees those requests. The frontend code is written the same way regardless (always same-origin `/api/*` fetches); only the physical proxying layer changes between dev and prod.
- A few *specific* non-`/api` paths are proxied via `next.config.js` `rewrites()` regardless of environment: `/api/docs*` and `/openapi.json` → backend (dev tooling), PostHog ingestion (`/ph_ingest/*` → PostHog's collector, keeping analytics same-origin for CSP/adblock reasons), and a Craft webapp HMR websocket passthrough.

Net takeaway for a new engineer: **write all backend calls as** `fetch("/api/ ... ")` and don't worry about the transport — dev and prod both resolve it to the FastAPI backend, just via different mechanisms.

3. Directory Structure (`web/src/`)

Directory	Convention
<code>app/</code>	Route tree only (Next.js App Router). <code>page.tsx</code> / <code>layout.tsx</code> files should be thin — they compose a <code>views/</code> component or directly render feature code. Some large features (chat, Craft) keep most of their implementation directly under their <code>app/</code> subtree instead of <code>views/</code> (see below) because they're not reused outside that one route.
<code>views/</code>	Top-level page implementations, one file/folder per admin or app page, e.g. <code>views/AgentEditorPage.tsx</code> , <code>views/admin/GroupsPage/</code> . A <code>views/admin/GroupsPage/index.tsx</code> is what <code>src/app/admin/groups/page.tsx</code> re-exports (<code>export { default } from "@views/admin/GroupsPage"</code>) — <code>app/</code> stays a pure routing shim, <code>views/</code> holds the actual page.
<code>components/</code>	Legacy, being phased out (per <code>web/AGENTS.md</code> : "Do NOT use anything from <code>web/src/components/</code> "). Still large (chat, admin, auth, sidebar, ui, tools, etc.) because migration is incremental, but new code should not add to it.
<code>refresh-components/</code>	The <i>current</i> production component layer, pending full migration into Opal. Organized by kind: <code>buttons/</code> , <code>cards/</code> , <code>inputs/</code> , <code>modals/</code> , <code>texts/</code> , <code>avatars/</code> , <code>layouts/</code> , <code>skeletons/</code> , <code>form/</code> (Formik-bound field wrappers like <code>InputTypeInField</code> , <code>PasswordInputTypeInField</code>), <code>contexts/</code> (e.g. <code>ModalContext</code>), <code>commandmenu/</code> , <code>popovers/</code> , <code>tiles/</code> . Prefer these over <code>components/</code> ; fall back to them only when Opal doesn't yet have the primitive.
<code>sections/</code>	Feature-specific composite components — the AGENTS.md guidance is "cards, modals, etc. that display a specific entity." Subfolders map to feature areas: <code>chat/</code> (<code>ChatUI.tsx</code>), <code>agents/</code> (<code>AgentCard.tsx</code> , <code>AgentsFilters.tsx</code>), <code>sidebar/</code> (<code>AppSidebar.tsx</code> , <code>AdminSidebar.tsx</code>), <code>admin/</code> (<code>AdminListHeader.tsx</code>), <code>input/</code> (the chat composer: <code>AppInputBar.tsx</code> , <code>BaseInputBar.tsx</code>), <code>model-selector/</code> , <code>modals/</code> , <code>cards/</code> , <code>document-sidebar/</code> , <code>knowledge/</code> , <code>onboarding/</code> , <code>projects/</code> , <code>settings/</code> , <code>skills/</code> , <code>banners/</code> .
<code>lib/</code>	Non-visual logic, organized by feature domain , not by kind — this is the most important convention to internalize. Each feature gets its own folder with a predictable file set: <code>svc.ts</code> (raw <code>fetch()</code> calls to the backend), <code>hooks.ts</code> (SWR-wrapped hooks built on <code>svc.ts</code>), <code>types.ts</code> / <code>interfaces.ts</code> . Examples: <code>lib/agents/{svc, hooks, types, utils, userPlaceholders}.ts</code> , <code>lib/chat/{svc, exportChatSession, fetchBackendChatSessionSS, greetingMessages}.ts</code> , <code>lib/auth/{components, hooks, svc, svcSS, types, utils, paths}.ts</code> , <code>lib/users/{svc, svcSS, hooks, types}.ts</code> , <code>lib/settings/{hooks, types}.ts</code> , <code>lib/connectors/</code> , <code>lib/credentials/</code> , <code>lib/billing/</code> , <code>lib/build/</code> (Craft), <code>lib/skills/</code> , <code>lib/tools/</code> , <code>lib/hierarchy/</code> , <code>lib/projects/</code> . A <code>svcSS.ts</code> suffix (e.g. <code>lib/users/svcSS.ts</code> , <code>lib/auth/svcSS.ts</code>) marks server-side-only variants used from Server Components/layouts (they can read the incoming request's cookies directly via <code>next/headers</code>), as opposed to the client <code>svc.ts</code> which uses the browser's ambient cookie jar. <code>lib/swr-keys.ts</code> is the single cross-cutting exception — a flat, centralized registry (see §4).
<code>hooks/</code>	Genuinely general-purpose hooks with no feature home — last resort per AGENTS.md's stated priority order (feature <code>lib/<x>/hooks.ts</code> first, then Opal, then here). Examples that legitimately belong here: <code>useScreenSize.ts</code> , <code>useClickOutside.ts</code> , <code>useFilter.ts</code> , <code>useDraft.ts</code> , <code>useWebSocket.ts</code> . Some feature-flavored hooks still live here for historical reasons (<code>useCCPairs.ts</code> , <code>useGroups.ts</code> , <code>useMcpServers.ts</code>) rather than under <code>lib/</code> .
<code>providers/</code>	React Context providers mounted near the app root. <code>AppProvider.tsx</code> is the composition root (see §4) — individual providers (<code>UserProvider.tsx</code> , <code>SettingsProvider.tsx</code> , <code>ProjectsContext.tsx</code> , <code>SWRConfigProvider.tsx</code> , etc.) each own one slice of global client state.
<code>interfaces.ts</code> (top-level, <code>src/interfaces/</code>)	Cross-cutting TypeScript types not owned by one feature.
<code>layouts/</code>	Page-level layout <i>primitives</i> (not full pages) — <code>layouts/chromes/</code> holds the outer app/admin chrome (<code>AppChrome.tsx</code> , <code>AdminChrome.tsx</code> , <code>AdminSSChrome.tsx</code> — the Server-Component auth-then-chrome wrapper), <code>layouts/general-layouts.tsx</code> exports low-level flex/section helpers (<code>Section</code> , etc.), <code>layouts/table-layouts.tsx</code> for table scaffolding, <code>layouts/craft/</code> for the Craft-specific shell, and <code>layouts/settings-layouts.tsx</code> (imported as <code>SettingsLayouts</code>) is the standard scaffold for every admin/settings page (<code>.Root</code> / <code>.Header</code> / <code>.Body</code>).

Directory	Convention
<code>ee/</code>	Enterprise-Edition-only code, mirroring the main tree's shape but tiny in this fork (<code>ee/lib/search/svc.ts</code> , <code>ee/providers/QueryControllerProvider.tsx</code> , <code>ee/sections/{SearchCard,SearchUI}.tsx</code> , <code>ee/views/admin/</code>). Only reached via the <code>proxy.ts</code> EE rewrite (\$2) — e.g. query history and usage-report pages under <code>/admin/performance/*</code> . Deliberately small in Orbyte because Groups (the biggest EE feature upstream) was pulled out into the MIT tier.
<code>refresh-components</code> vs <code>sections</code> vs <code>views</code> vs <code>lib</code> — mental model	<code>lib/</code> = logic and data, no JSX. <code>refresh-components/</code> = generic, reusable, no domain knowledge (a <code>Button</code> , a <code>Modal</code>). <code>sections/</code> = domain-aware composite UI (an <code>AgentCard</code> , a <code>GroupsList</code>) built from <code>refresh-components</code> /Opal. <code>views/</code> = a full page, composed from <code>sections/</code> + <code>lib/</code> hooks, plugged into <code>app/</code> by a one-line re-export.

4. State Management & Data Fetching

SWR is the fetching layer; `lib/swr-keys.ts` is the key registry

`web/src/lib/swr-keys.ts` exports a single `SWR_KEYS` const object (230 lines) grouped by feature with comment-banner sections (`Groups`, `Users`, `Agents / Personas`, `Build (Craft)`, `Query History (EE)`, etc.). Two shapes appear:

- Static string keys for fixed endpoints: `me: "/api/me"`, `adminUserGroups: "/api/manage/admin/groups"`.
- Builder functions for parameterized endpoints: `groupJoinLinks: (groupId: number) => "/api/manage/admin/groups/${groupId}/join-links"`, `joinLinkLookup: (token: string) => "/api/join-link/${token}"`.

The stated purpose (doc comment at the top of the file) is to prevent typos and keep key usage greppable — every `useSWR( ... )` call and every `mutate( ... )` cache-invalidation call should reference `SWR_KEYS.x` rather than an inline string literal, so a single `grep` for a key name finds every read *and* every place that invalidates it.

The `svc.ts` + `hooks.ts` pairing

The dominant pattern throughout `lib/<feature>/`:

- `svc.ts` — plain async functions wrapping `fetch()`, no React. Each function does one mutation or one-off read, throws a normal `Error` with a backend-supplied `detail` message on failure, and returns parsed JSON (or `void`) on success. Example, `lib/agents/svc.ts`:

```
export async function parseErrorDetail(res: Response, fallback: string) {
  try {
    const body = await res.json();
    return typeof body?.detail === "string" ? body.detail : fallback;
  } catch {
    return fallback;
  }
}
```

(`views/admin/GroupsPage/svc.ts` follows the identical shape — see §5 for the full walkthrough.)

- `hooks.ts` — wraps `useSWR` / `useSWRConfig` around the `svc.ts` functions and the feature's SWR keys, exposing a ready-to-consume hook. Example, `lib/agents/hooks.ts`:

```
export function useAgents() {
  const { data, error, mutate } = useSWR<MinimalAgent[]>(
    SWR_KEYS.personas,
    errorHandlerFetcher,
    { revalidateOnFocus: false, revalidateIfStale: false, dedupingInterval: 60000 }
  );
  return { agents: data ?? [], isLoading: !error && !data, error, refresh: mutate };
}
```

Every read-only hook uses the shared `errorHandlingFetcher` from `src/lib/fetcher.ts` (throws a typed `FetchError` / `RedirectError` with `.status`, so callers can branch on 401/403/etc. — see `useGroupMemberCandidates.ts`'s use of `skipRetryOnAuthError` to stop SWR retry-storming a tier-gated 402).

A second concrete example is `views/admin/GroupsPage/useGroupMemberCandidates.ts`: it composes two SWR calls (`SWR_KEYS.groupMemberCandidates`, conditionally `SWR_KEYS.adminApiKeys` gated on `isAdmin` — passing `null` as the SWR key is the idiom for "don't fetch this"), then derives/memoizes a combined `MemberRow[]` list. This is the standard shape for a hook that needs to merge multiple backend calls into one UI-ready shape.

Global Context providers

Composition root: `src/providers/AppProvider.tsx`, mounted once near the root layout. Its own doc comment documents the intentional nesting order (outermost → innermost):

1. **SettingsProvider** (`src/providers/SettingsProvider.tsx`) — app-wide settings/feature flags. Actual reads go through `useSettings()` in `src/lib/settings/hooks.ts`, which merges two SWR calls (`/api/settings` core settings, `/api/enterprise-settings`) into one `AppSettings` object with derived fields like `appName` (falls back to "Orbyte"), `logoUrl` (cache-busted with a timestamp query param so a freshly-uploaded logo isn't served stale from browser cache), and `vectorDbEnabled`. It intentionally skips the core-settings fetch on `/auth/*` paths (unauthenticated there) and skips the enterprise-settings fetch once it's determined core settings say EE is off.
2. **UserProvider** (`src/providers/UserProvider.tsx`) — the current user object, `isAdmin`, `isCurator` (true for either `CURATOR` or `GLOBAL_CURATOR` roles — this is what gates the Curator toggle in Groups, §7), `authTypeMetadata` (password policy, SSO metadata), and a large set of `updateUserX` mutators (theme preference, chat background, default model, pinned agents, auto-scroll, etc.) that each optimistically update local state then PATCH the backend and roll back via `refreshUser()` on failure. Wraps `useCurrentUser()` (SWR on `/api/me`) and layers in theme sync (via `next-themes`) and PostHog `identify()` / `reset()` calls. Almost every feature component in the app reads from `useUser()`.
3. **AppBackgroundProvider** — resolves the chat background image URL from user preference.
4. **ProviderContextProvider** (`src/components/chat/ProviderContext.tsx`) — LLM provider config available to chat/agent UI.
5. **ModalProvider** (`src/components/context/ModalContext.tsx`, plus a newer `refresh-components/context/ModalContext.tsx` used via `useCreateModal()`) — centralized modal stack so any component can open a modal without prop-drilling.
6. **SidebarPersistenceProvider** (local to `AppProvider.tsx`) → Opal's **SidebarStateProvider** — persists the sidebar folded/expanded state to both a cookie and `localStorage` (cookie so a Server Component can theoretically read it, `localStorage` as the client source of truth).
7. **QueryControllerProvider** — chat vs. search mode and query lifecycle state.
8. **FullWidthChatProvider** — the "full width chat" display toggle.
9. **ToastProvider** (Opal) — global toast/notification host, innermost so `toast.success()` / `toast.error()` (used everywhere, e.g. throughout `JoinLinksSection.tsx` / `EditGroupPage.tsx`) work from any descendant.

Separately, `src/providers/SWRConfigProvider.tsx` wraps the tree in `<SWRConfig value={{ onErrorRetry: skipRetryOnAuthError }}>` — this sets the **global default** retry behavior (stop retrying on 401/402/403) so every `useSWR` call gets sane auth-aware retry semantics unless it opts out.

Zustand for non-SWR state: `src/app/app/stores/useChatSessionStore.ts` is a Zustand store (not SWR) for the active chat session's message tree/history/loading-error state — chosen because chat streaming needs frequent, fine-grained, non-React-render-driven updates (packets arriving over SSE) that don't map cleanly onto SWR's cache-and-revalidate model.

5. API Communication Layer

The `svc.ts` convention, in full, using Groups as the reference example

`src/views/admin/GroupsPage/svc.ts` is a clean, representative example. Every exported function follows the same three-step shape: **fetch** → **check** `res.ok` → **throw or return**:

```

async function renameGroup(groupId: number, newName: string): Promise<void> {
  const res = await fetch(`${USER_GROUP_URL}/rename`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ id: groupId, name: newName }),
  });
  if (!res.ok) {
    const detail = await res.json().catch(() => null);
    throw new Error(detail?.detail ?? `Failed to rename group: ${res.statusText}`);
  }
}

```

Notes on the pattern:

- `USER_GROUP_URL` is derived from `SWR_KEYS.adminUserGroups` rather than hardcoded — even mutation endpoints reuse the SWR key registry as their base URL where they share a path prefix with a read.
- Error handling is uniform: attempt to parse the JSON error body, prefer the backend's `detail` field, fall back to a generic message built from `res.statusText`. The `.catch(() => null)` guards against a non-JSON error body (e.g. a raw 502 from a proxy).
- Callers (e.g. `EditGroupPage.tsx`'s `handleSave`) catch with `e instanceof Error ? e.message : "..."` and feed the message straight into `toast.error(...)`. There's no centralized global error boundary for API errors — each call site is expected to catch and toast.
- More complex mutations sometimes have to **read-modify-write** because the backend has no dedicated "add group to resource" endpoint — see `updateDocSetGroupSharing` in the same file: it fetches all document sets, finds the ones whose `groups` array changed, and re-PATCHes the whole document-set object with the updated `groups` array spliced in. This is a useful pattern to recognize: not every relationship has its own PATCH endpoint, so the frontend sometimes owns the diffing logic.
- Bulk/batch operations that don't have a bulk endpoint are simply done as a sequential loop of individual REST calls (see `saveTokenLimits`'s three loops: update existing-by-index-position, create new, delete excess).

Error handling primitives (`src/lib/fetcher.ts`)

- `FetchError` / `RedirectError` — typed errors carrying `.status` and `.info` (parsed JSON body), thrown by the shared `errorHandlingFetcher` used as the SWR fetcher almost everywhere.
- `skipRetryOnAuthError` — an `onErrorRetry` callback (set as the global SWR default via `SWRConfigProvider`, and passed explicitly to individual `useSWR` calls that need it, e.g. the group token-rate-limit fetch in `EditGroupPage.tsx`) that immediately gives up retrying on 401/402/403 (auth or tier-gating errors — retrying those is pointless and just spams the backend), but keeps exponential backoff (capped at 30s) for genuine transient errors. It also pauses retries while the tab is hidden and resumes on `visibilitychange`, to avoid burning retry budget on backgrounded tabs.

Auth / session handling: cookies, not tokens, and the browser never touches the token

- `basicLogin(email, password, captchaToken)` (`src/lib/users/svc.ts`) POSTs `application/x-www-form-urlencoded` `username / password` to `/api/auth/login` with `credentials: "include"`. This is a FastAPI-Users-style cookie-based login: the backend responds with a `Set-Cookie` header (the cookie name is configured via `SERVER_SIDE_ONLY__AUTH_COOKIE_NAME`, referenced in both `proxy.ts` and `src/app/api/[... path]/route.ts`). The frontend never reads or stores a bearer token — the session is carried entirely by that (HttpOnly, so JS-inaccessible) cookie on every subsequent same-origin request.
- `logout()` POSTs to `/auth/logout` (a Route Handler at `src/app/auth/logout/route.ts`, not the catch-all proxy) and, on success, does `mutate(SWR_KEYS.me, null, { revalidate: false })` to immediately clear the cached `/api/me` SWR entry — otherwise a component that re-renders during the dedupe window could momentarily show the just-logged-out user.
- `getCurrentUser()` is just `fetch("/api/me", { credentials: "include" })`; a 404/non-OK response is treated as "no user," not an exception.
- Server-side equivalents (`src/lib/users/svcSS.ts`, `src/lib/auth/svcSS.ts`) exist for Server Components — they read the incoming request's cookie header directly (via `next/headers`) and call the backend with it forwarded, since a Server Component has no browser cookie jar to rely on implicitly. `requireAuth()` / `requireAdminAuth()` (used in `app/app/layout.tsx` and `AdminSSChrome.tsx` respectively) are built on this.
- Token **refresh** (for auth types that use short-lived tokens, e.g. some SSO configs) is handled by `refreshToken(customRefreshUrl)` in `lib/users/svc.ts` plus `useTokenRefresh()` (wired into `UserProvider`), which on failure falls back to `mutateUser()` (re-fetch

`/api/me` and let the normal SWR error path take over — effectively "if refresh fails, find out you're logged out the next time something reads the user").

- CAPTCHA: several auth flows (`basicLogin`, `basicSignup`, the Join Link redemption flow) optionally attach an `X-Captcha-Token` header, sourced from `useCaptcha().getCaptchaToken(action)` — a reCAPTCHA v3 token fetched only when `NEXT_PUBLIC_RECAPTCHA_SITE_KEY` is configured at build time.

6. Component Library / Design System

Opal (`@onyx-ai/opal`, `web/lib/opal/src/`)

Opal is described in its own `package.json` as "Opal — Onyx's Typescript component library and design system." It's a real local package (built via `tsup`, `bun run prepare` on install) with subpath exports: `@opal/components`, `@opal/layouts`, `@opal/core`, `@opal/icons`, `@opal/illustrations`, `@opal/logos`, `@opal/types`, `@opal/utils`, plus a `tailwind-preset` and `root.css` that the app's own Tailwind config and global stylesheet build on top of.

Structure under `lib/opal/src/`: `components/` (buttons, cards, checkbox, code, inputs, pagination, popover, table, tabs, tag, text, tooltip, calendar, divider, spacer, shadow-div, end-of-list), `core/` (the lowest-level primitives — see below), `layouts/` (page-level layout scaffolding: `auth/`, `cards/`, `content/`, `content-action/`, `general/`, `illustration-content/`, `inputs/`, `root/`, `sidebar/`, `settings/`, `tag-list/`, `toast/`), `hooks/`, `icons/`, `illustrations/`, `logos/`.

Components actually observed imported from `@opal/components` across the codebase (import-count from a repo-wide grep, most-used first): **Button** (205 uses), **Text** (129), **InputTypeIn** (50), **Divider** (40), **Tooltip** (33), **Popover/PopoverMenu** (44), **Checkbox** (18), **Spacer** (17), **CopyButton** (17), **Switch** (16), **MessageCard** (15), **Card** (14), **ListItemButton** (10), **Tabs** (9), **SidebarTab** (9), **Tag** (8), **EmptyMessageCard** (8), **Table/createTableColumns** (5), **SelectButton**, **ShadowDiv**, **SelectCard**, **FilterButton**, **Pagination**, **Code**, **CompactMarkdown**. From `@opal/layouts`: `SettingsLayouts`, `AuthLayouts`, `Content / ContentAction / IllustrationContent`, `InputVertical / InputHorizontal`, `RootLayout`, `SidebarLayouts`, `toast`, `useSidebarState`, `ToastProvider`, `SidebarStateProvider`. From `@opal/core`: `Interactive` (the color/state matrix underlying every clickable surface), `Disabled` (a Radix-`Slot`-based disabled-visuals wrapper), `Hoverable` (coordinated hover-to-reveal, replacing manual `opacity-0 group-hover:opacity-100` Tailwind).

Key convention (from `web/AGENTS.md`): `web/src/components/` is legacy and off-limits for new code; prefer Opal first, fall back to `refresh-components` for anything Opal doesn't yet cover. `SettingsLayouts.Root / .Header / .Body` is the mandated scaffold for every admin/settings page — `GroupsPage`, `EditGroupPage`, and essentially every other `views/admin/*` page all open with it (confirmed directly in `EditGroupPage.tsx` and `GroupsPage/index.tsx`).

Theming / color tokens

The token pipeline is: `web/lib/shared/tokens/*.json` (Style-Dictionary source: `primitives.json` — raw grey/alpha color ramps like `--grey-04`, `--alpha-grey-100-55` — plus `semantic-light.json` and `semantic-dark.json`, which alias primitives to semantic names per theme, e.g. `"text-05": { "value": "{alpha-grey-100-90}" }` in light mode) → compiled by Style Dictionary into `web/lib/shared/dist/tokens.css` → imported into Opal's `_reference.css / root.css` → pulled into the app's global stylesheet. `web/tailwind-themes/tailwind.config.js` then maps Tailwind utility classes onto these CSS variables (`darkMode: "class"`, presets from `@onyx-ai/opal/tailwind-preset`), so e.g. `bg-background-neutral-01` resolves to `var(--background-neutral-01)`, which flips value automatically between light/dark via the semantic token files — **not** via Tailwind's `dark:` variant.

This is why `web/AGENTS.md` states the hard rule: "Strictly forbid using the `dark:` modifier in Tailwind classes, except for logo icon handling" — the semantic color tokens already encode the dark-mode-appropriate value, so a component that just uses `bg-background-neutral-01 / text-text-03` / etc. is automatically theme-correct; sprinkling `dark:` on top would fight the token system and (per the doc) "lead to horrible UI breakages." The color categories available: `text-01..05` (+ `text-inverted-XX`), `background-neutral-XX / background-tint-XX` (+inverted), `border-01..05` (+inverted), `action-link-XX / action-danger-XX`, `status-info/success/warning/ error-XX`, `theme-primary-XX / theme-red-XX` /etc. Icons are equally locked down: only `web/src/icons/` may be imported (no `lucide-react / react-icons` in new code — those packages appear in `package.json` only as transitive/legacy leftovers).

7. Major Feature Areas

7.1 Chat interface (/app)

Entry point: `src/app/app/page.tsx` → `src/views/AppPage.tsx` (1087 lines — the main chat page component). Layout gating (`src/app/app/layout.tsx`) is a Server Component that calls `requireAuth()`, then mounts `<ProjectsProvider><RootLayout.Root><AppSidebar /><AppChrome> {children}</AppChrome></RootLayout.Root></ProjectsProvider>`.

Core building blocks, all under `src/app/app/`:

- Message list / rendering:** `message/` holds `HumanMessage.tsx`, and `message/messageComponents/AgentMessage.tsx` for the assistant side. Streaming responses are decomposed into a **timeline of packets** — `message/messageComponents/timeline/` and `message/messageComponents/renderMessageComponent.tsx` dispatch each packet type to a dedicated renderer under `message/messageComponents/renderers/` (e.g. `ImageToolRenderer.tsx`, `CustomToolRenderer.tsx`). `services/packetUtils.ts` and `services/currentMessageFIFO.ts` handle the actual SSE stream parsing/sequencing; `MemoizedTextComponents.tsx` and `messageComponents/markdownUtils.tsx` handle the Markdown→HTML rendering pipeline (with citation handling, code blocks via `CodeBlock.tsx`, KaTeX, etc.).
- Multi-model comparison:** `MultiModelPanel.tsx` / `MultiModelResponseView.tsx` / `multiModel.ts` — lets a user compare responses from more than one model side-by-side for the same prompt.
- Message tree / branching:** `services/messageTree.ts` models the fact that a chat is a tree (edits/regenerations create branches), with `MessageSwitcher.tsx` as the UI for navigating siblings and `Resubmit.tsx` for regeneration.
- Input box:** `src/sections/input/AppInputBar.tsx` (built on `BaseInputBar.tsx`) is the composer — includes `PlusMenuButton.tsx` (attach files / tool toggles), `InputChipStrip.tsx` (attached file/context chips), `EntryPickerPopover.tsx` / `EntryInfoPopover.tsx`, and `QueuedMessageBar.tsx` (messages queued while a response is still streaming).
- Model/persona (agent) selector:** `src/sections/model-selector/` (`ModelSelector.tsx`, `MultiModelSelector.tsx`).
- Session state:** `src/app/app/stores/useChatSessionStore.ts` (Zustand — current message tree, history, loading/error flags) is read via selector hooks like `useCurrentMessageHistory()`, `useCurrentMessageTree()`, `useLoadingError()`, `useUncaughtError()` (all exported from that same store file and consumed directly by `ChatUI.tsx`).
- ChatUI.tsx** (`src/sections/chat/ChatUI.tsx`) is the actual message-list renderer component — it's what turns the store's message tree + streaming state into the rendered conversation, independent of the input bar and sidebar.
- Data layer: `src/lib/chat/svc.ts`, `fetchBackendChatSessionSS.ts` (server-side session fetch for the shared-chat page), `exportChatSession.ts` (chat export), `greetingMessages.ts`.
- Shared chat links:** `src/app/app/shared/[chatId]/page.tsx` + `SharedChatDisplay.tsx` — a read-only public rendering of a shared chat transcript.

7.2 Agents / Personas

"Agents" is the current name for what upstream Onyx calls "Personas" (the DB/type layer still says `Persona` in many places — e.g. `SWR_KEYS.personas`, `Agent` / `FullAgent` / `MinimalAgent` types extend the persona concept). Legacy `/admin/assistants` and `/ee/assistants/*` URLs 301-redirect to `/admin/agents` / `/ee/agents/*` respectively (see `next.config.js` `redirects()`) — kept for old bookmarks per the tracked TODO to remove in v4.0).

- Editor:** `src/views/AgentEditorPage.tsx` is the single create/edit form (used by both `src/app/app/agents/create/page.tsx` and `src/app/app/agents/edit/[id]/`). It's a large Formik form (`Formik` / `Form` / `FieldArray` from `formik`, `Yup` schema) covering: name/description, system+task prompts (with `InsertUserVariableMenu.tsx` for `{{variable}}` placeholders), avatar (`InputAvatar.tsx`, `CustomAgentAvatar.tsx`), starter messages, model selection (`ModelSelector.tsx`), knowledge/document-set attachment (`AgentKnowledgePane.tsx`, `useDocumentSets()`), tool toggles (web search, URL-opening, search — via `SwitchField` / constants from `app/app/components/tools/constants.ts`), MCP tool selection (`useMcpServersForPersonaEditor`, `useAvailableTools`), sharing (`ShareAgentModal.tsx`, `updateAgentShares`), labels (`useLabels`), and delete/soft-delete (`deleteAgent`, `toggleAgentListed`).
- Data layer:** `src/lib/agents/{svc, hooks, types, utils, userPlaceholders}.ts`. `svc.ts` builds the wire-format `AgentUpsertRequest` from a client-facing `AgentUpsertParameters` object (a translation-layer function, `buildAgentUpsertRequest`, deliberately hardcodes `display_priority: null` since ordering is managed by a separate endpoint). `hooks.ts` layers SWR on top (`useAgents()` — 60s dedupe, no focus-revalidate, since the agent list rarely changes mid-session).
- Listing/browsing:** `src/app/app/agents/page.tsx` + `src/sections/agents/AgentCard.tsx` / `AgentsFilters.tsx` (has a `README.md` documenting the filter conventions).

- **Admin overview page:** `src/app/admin/agents/page.tsx` (admin-level list/management, distinct from the end-user `/app/agents` browsing page) plus `src/lib/swr-keys.ts`'s `adminAgents` / `adminPersona` keys.

7.3 Admin panel overview

Entry: `src/app/admin/layout.tsx` (Server Component) → `AdminSSChrome` (`requireAdminAuth()`), then `AdminChrome` .
`AdminChrome.tsx` renders `AdminSidebar` (unless the current route declares its own custom sidebar — currently only `/admin/connectors/*`) plus a payment-reminder banner keyed off `ApplicationStatus.PAYMENT_REMINDER` , and gates any `isVectorDbRequiredRoute` page behind a "Lite Mode" notice (`LiteModeIndexingNotice`) if no vector DB is configured.

`AdminSidebar.tsx` (`src/sections/sidebar/AdminSidebar.tsx`) builds its nav items from a typed route registry (`ADMIN_ROUTES` / `sidebarItem()`) in `src/lib/admin-routes.ts`), grouped into labeled sections: *Agents & Actions*, *Documents & Knowledge*, *Permissions*, plus a few unlabeled top items (LLM Models, Web Search, Chat Preferences) for admins only. Notably, the sidebar-builder code contains explicit comments marking **removed** upstream sections that are intentionally not wired up: "OpenAPI Actions: disconnected from UI per STRIPPING_METHODODOLOGY.md Phase 2," "Service Accounts: disconnected from UI," and "Query History disconnected from UI" (Query History still exists as an EE-gated route reachable directly, just not linked from the sidebar in this build). Curators (`isCurator === true`) see a reduced item set — just Agents, Documents/Knowledge, and Groups (no LLM/Web Search/Users, since those are core-admin-only).

Top-level admin areas actually present under `src/app/admin/` : `agents` , `bots` (Slack bot config), `configuration/` (`chat-preferences` , `code-interpreter` , `craft` , `document-processing` , `index-settings` , `language-models` , `web-search`), `connector/` `connectors` (connector CRUD), `craft/` (`access` , `apps` , `instructions` — Craft admin controls), `discord-bot` , `documents/` (`explorer` , `feedback` , `sets`), `federated` , `groups/` `groups2` (see §7.4 — `groups2` is a duplicate route mounted to the exact same `GroupsPage` view, likely a staged-rollout artifact), `indexing/status` , `oauth-test` , `security` , `systeminfo` , `token-rate-limits` , `tracing` , `users` , plus a `debug` page.

7.4 Groups & Curators (NEW — Orbyte-specific)

This is a genuinely new feature relative to upstream Onyx, and it's now first-party/MIT-tier (not EE-gated — see the `proxy.ts` comment quoted in §2). All code lives under `web/src/views/admin/GroupsPage/` .

File map:

- `index.tsx` — the group **list** page (`GroupsPage`). Fetches `SWR_KEYS.adminUserGroups` , renders an info banner about upcoming permission changes, an `AdminListHeader` (search + "New Group" action wired to `/admin/groups/create`), and `GroupsList` / `GroupCard` for the actual list.
- `CreateGroupPage.tsx` — the create form.
- `EditGroupPage.tsx` — the create/edit form's bigger sibling; this is the heart of the feature (556 lines).
- `svc.ts` — all group mutations (see §5 for the pattern walkthrough).
- `joinLinkSvc.ts` / `JoinLinksSection.tsx` — the Join Links sub-feature (§7.5).
- `useGroupMemberCandidates.ts` — the member-picker data hook.
- `SharedGroupResources/` — the agent/doc-set/connector sharing widget embedded in the edit page.
- `TokenLimitSection.tsx` — Enterprise-tier token rate limits for the group.
- `shared.tsx` , `utils.ts` , `interfaces.ts` — column defs (`baseColumns` , `memberTableColumns` , `tc` table-column helper, `PAGE_SIZE`), utility functions, and the `MemberRow` / `TokenRateLimitDisplay` / `ApiKeyDescriptor` types.

`EditGroupPage.tsx` walkthrough (`groupId: number` prop, routed from `src/app/admin/groups/[id]/page.tsx`):

1. **Data loading:** `useSWR(SWR_KEYS.adminUserGroups, ...)` fetches *all* groups (not a single-group endpoint) and finds the current one by ID client-side — with a clever `refreshInterval` function that polls every 5s **only while this specific group's `is_up_to_date` flag is false** (i.e. while a backend embedding-resync job is still running after a membership/doc-set change), and stops polling once synced. A separate `useSWR(SWR_KEYS.userGroupTokenRateLimit(groupId), ...)` fetches Enterprise-tier token limits, using `onErrorRetry: skipRetryOnAuthError` so a 402 (wrong tier) doesn't retry-storm.
2. **Member management:** Two view modes toggle via `isAddingMembers` state — a read-only "current members" `Table` (from `@opal/components`) vs. an "add mode" `Table` showing *all* candidate users/service-accounts (from `useGroupMemberCandidates()`) with multi-select (`selectionBehavior="multi-select"` , `initialRowSelection` seeded from current members). A subtlety documented in-code: `hiddenMemberIds` preserves already-selected members who don't appear in the currently-loaded candidate rows (e.g. inactive users), so toggling into "add mode" and back doesn't silently drop them from the selection when `onSelectionChange` fires with only the visible rows.

3. **The Curator toggle** — this is the actual "per-group Curators" feature: the members table's column definition conditionally injects a **"Curator" column** only `... (isAdmin ? [ ... ] : [])` — i.e. **only full admins can see or change curator status**; a curator viewing their own group (curators can be given access to manage groups they curate) never even sees the toggle control. The column cell renders an Opal `Switch` bound to `selectedCuratorIds.includes(userId)`, toggled via `handleToggleCurator`. Curator IDs are tracked as a separate `selectedCuratorIds` array from plain membership (`selectedUserIds`) — a user must be a member to be a curator, but not every member is a curator. Removing a member (`handleRemoveMember`) also strips them from `selectedCuratorIds`. On save, `updateGroup(groupId, selectedUserIds, selectedCcPairIds, selectedCuratorIds)` PATCHes `/api/manage/admin/groups/{id}` with a `curator_ids` field. Elsewhere in the codebase (`UserProvider.tsx`), a user's *global* role can independently be `CURATOR` / `GLOBAL_CURATOR`, and `isCurator` is true for either — group-level curator status (this feature) is a separate, per-group grant layered on top of that role system, letting a given user be a curator for group A but not group B.
4. **Agent / doc-set / connector sharing**: delegated to `<SharedGroupResources>` (own subdirectory: `ResourceContent.tsx` / `ResourcePopover.tsx`), driven by three parallel `selected*Ids` state arrays (`selectedCcPairIds`, `selectedDocSetIds`, `selectedAgentIds`) seeded from the group's `cc_pairs` / `document_sets` / `personas` on load. Saving only sends **diffs**: `initialAgentIdsRef` / `initialDocSetIdsRef` snapshot the starting IDs, and `svc.ts`'s `updateAgentGroupSharing` / `updateDocSetGroupSharing` compute added/removed sets and only touch what changed (agents have a dedicated `/groups/{id}/agents` PATCH endpoint; doc sets don't, so that function falls back to the read-modify-write pattern described in §5).
5. **Token rate limits** (`TokenLimitSection`) — Enterprise-only (`disabled={!isEnterpriseTier}`), tier check via `tierAtLeast(settings.tier, Tier.ENTERPRISE)`, with a markdown-linked tooltip pointing at `/admin/billing` when locked.
6. **Join Links** (`<JoinLinksSection groupId={groupId} />`) — embedded directly in the edit page; see §7.5.
7. **Save flow** (`handleSave`): re-fetches the group list fresh (bypassing SWR cache) right before submitting, specifically to re-check `is_up_to_date` and block saving with a toast if a sync is still in flight (prevents racing the backend's resync job with a second edit). Then sequentially: rename (if changed) → `updateGroup` (members/cc-pairs/curators) → agent-sharing diff → doc-set- sharing diff → token limits (if Enterprise) → `mutate()` both SWR keys → success toast → redirect to `/admin/groups`.
8. **Delete**: a `ConfirmationModalLayout` gates `deleteGroup(groupId)`, warning that members lose access to anything shared only with this group.

7.5 Join Links (NEW — Orbyte-specific)

Replaces upstream Onyx's email-invite flow with **group-scoped, shareable signup links**. Three files/pages implement it end-to-end:

Admin side — `JoinLinksSection.tsx` (embedded inside `EditGroupPage.tsx`, wrapped in a `SimpleCollapsible` labeled "Join Links" with the description "Shareable links that let new members self-register directly into this group — no email required.):

- Lists existing links via `useSWR(SWR_KEYS.groupJoinLinks(groupId), ...)`, showing per-link reusability (`is_reusable` → "Reusable"/"Single-use"), use count, expiry date (`formatDate(link.expires_at)`, "Never" if null), and a revoked state (grayed text, no delete button once `revoked_at` is set).
- **Create flow**: a modal lets the admin pick `isReusable` (Switch) and an expiry from fixed options (Never / 24h / 7 days / 30 days — `EXPIRY_OPTIONS`). Submitting calls `createJoinLink(groupId, isReusable, expiryHours)` (`joinLinkSvc.ts`), which POSTs to `SWR_KEYS.groupJoinLinks(groupId)` with `{ is_reusable, expires_in_hours }` and returns `{ id, token, is_reusable, expires_at }`.
- **Reveal-once modal**: immediately after creation, a *second* modal opens showing `${window.location.origin}/join/${createdLink.token}` with a `CopyButton`, and the explicit copy "Copy this link now — it won't be shown again." This is a deliberate one-time-reveal UX — the raw token is never re-displayed after this modal closes (the list view only ever shows metadata, never the token/URL, consistent with a bearer-token-style credential).
- **Revoke**: `revokeJoinLink(groupId, linkId)` issues a `DELETE` to `${SWR_KEYS.groupJoinLinks(groupId)}/${linkId}`; the UI just re-fetches the list (`mutate()`) and the row flips to its revoked visual state rather than disappearing (preserving audit history of past links).

`joinLinkSvc.ts` — the `svc.ts` for this sub-feature; exports `createJoinLink` / `revokeJoinLink` plus the `JoinLinkSnapshot` / `JoinLinkCreateResponse` types, following the exact same fetch-check-throw pattern described in §5.

Public side — `src/app/join/[token]/page.tsx`: an async Server Component. It attempts `getCurrentUserSSC()` (server-side, cookie-forwarding user lookup); if there's already an active, non-anonymous logged-in user, it `redirect("/app")` immediately — the reasoning in the code comment: "Someone already logged in shouldn't redeem a link meant for a new teammate." Otherwise it renders `<AuthFlowContainer authState="signup"><JoinLinkForm token={token} /></AuthFlowContainer>` — reusing the same auth-page chrome as login/signup.

`JoinLinkForm.tsx` (Client Component, the actual redemption form):

1. `useSWR(SWR_KEYS.joinLinkLookup(token), ...)` validates the token up front (`{ valid, group_name }`) — an invalid/expired/already-used link renders an "Invalid link" message instead of a form (no separate error page needed).
2. A **Formik + Yup** form (email + password) whose password schema is built dynamically from `authTypeMetadata` (`passwordMinLength` / `Max` / `RequireUppercase` /etc., the same policy object `UserProvider` exposes app-wide) — this is exactly the shared password-policy-builder pattern the task brief calls out in `lib/auth/components.tsx`'s `PasswordRequirements` (reused verbatim here via the same helper predicates: `passwordHasUppercase`, `passwordHasLowercase`, `passwordHasDigit`, `passwordHasSpecialChar`, `passwordMeetsLengthRequirements`, all from `lib/auth/utils.ts`), and the same `<PasswordRequirements password={values.password} />` live checklist component is reused here unchanged.
3. **Submit**: fetches a signup-action CAPTCHA token (`getCaptchaToken("signup")`), then POSTs directly to `/api/join-link/${token}/redeem` (not through `joinLinkSvc.ts` — this is a one-off inline fetch since it's public/unauthenticated) with `{ email, password }` and `credentials: "include"`. Specific status-code handling: `410` → "invalid or expired" (the backend's way of signaling the link can no longer be redeemed), `429` → rate-limited message, `REGISTER_USER_ALREADY_EXISTS` detail → "account already exists" message.
4. **Hand-off into `basicLogin`**: on a successful redemption response, it immediately calls `basicLogin(email, values.password, loginCaptchaToken)` (the exact same function used by the normal login page, `lib/users/svc.ts`) to establish the session cookie, then `window.location.href = "/app"` on success — a full navigation (not client-side router push), ensuring the freshly-set auth cookie is picked up cleanly by the next full page load. If the auto-login call fails for some reason, it falls back to toasting "Account created — please log in with your new credentials" and sending the user to `/auth/login` instead of leaving them stuck.

This flow is a clean illustration of composition: a brand-new, product-specific feature (group-scoped signup links) was built almost entirely by **reusing** existing auth primitives (`basicLogin`, `authTypeMetadata` password policy, `PasswordRequirements`, `AuthFlowContainer`) — only the token-lookup SWR key and the one bespoke `/redeem` POST are new.

7.6 Connectors (brief pass)

Admin UI lives under `src/app/admin/connectors/` and `src/app/admin/connector/`. The dynamic `[connector]` route (`AddConnectorPage.tsx`, `ConnectorWrapper.tsx`, `NavigationRow.tsx`) drives a per-source-type connector setup wizard, with a nested `oauth/` segment for connectors that need an OAuth handshake (Google Drive/Gmail, etc. — see `SWR_KEYS.googleConnectorCredentials` / `googleConnectorPublicCredential` / `googleConnectorServiceAccountCredential` for the credential flows) and `auth/` / `pages/` subfolders for connector-specific config forms. `/admin/connector/[ccPairId]` is the per-connector-instance status/detail page (indexing status, errors — see `SWR_KEYS.ccPairIndexingErrors`, `ccPairPermissionSyncAttempts`, `ccPairExternalGroupSyncAttempts`). Data hooks: `src/hooks/useCCPairs.ts`, `src/lib/connectors/`, `src/lib/credentials/`, `src/lib/indexing/`. `AdminChrome.tsx` special-cases `/admin/connectors/*` to render its own sidebar instead of the standard `AdminSidebar`.

7.7 Craft (code execution / build UI) — brief pass

Craft is a large, self-contained feature under `src/app/craft/` (its own `README.md`) — an agentic "build" experience (chat-driven code/app generation with a sandboxed execution environment, live approvals, scheduled tasks, and a file/output preview panel). Key pieces: `components/ChatPanel.tsx` + `BuildMessageList.tsx` (the build-chat transcript), `OutputPanel.tsx` (live app/file preview), `contexts/BuildContext.tsx`, `hooks/useBuildSessionController.ts` + `useBuildStreaming.ts` (its own SSE-driven streaming stack, parallel to but separate from the main chat's), `utils/parsePacket.ts` (Craft has its own packet-stream protocol, distinct from `app/app/services/packetUtils.ts`), and `services/apiServices.ts` / `externalAppsService.ts`. Admin controls live at `src/app/admin/craft/{access,apps,instructions}` (who can use Craft, registered external apps, base system instructions). `SWR_KEYS` "Build (Craft)" section shows the endpoint surface: session file trees, artifacts, webapp info, scheduled tasks/runs, live approvals. `/craft/v1` is a second, older/parallel route also present (`layout.tsx`, `page.tsx`, `apps/`, `skills/`, `tasks/`) — worth a closer look before touching Craft routing, as it's unclear from this pass alone whether `v1` is legacy-being-retired or a stable alternate entry point.

7.8 Query History (brief pass)

Lives entirely under the EE tree: `src/app/ee/admin/performance/query-history/ QueryHistoryTable.tsx`. It's one of the four routes explicitly listed in `proxy.ts`'s `EE_ROUTES` (`/admin/performance/query-history`), so it's only reachable when `SERVER_SIDE_ONLY_PAID_ENTERPRISE_FEATURES_ENABLED` is set, at which point `proxy.ts` transparently rewrites `/admin/performance/query-history` → `/ee/admin/performance/query-history`. As noted in §7.3, `AdminSidebar.tsx` currently does **not** link to it (marked "disconnected from UI" in that file's comments) — it exists in code and is reachable by direct URL on an EE build, but isn't promoted in the nav for this product. `SWR_KEYS.adminChatSessionHistory` / `adminChatSession(id)` back it.

7.9 Auth pages

All under `src/app/auth/`: `login/` (`LoginPage.tsx`, `LoginText.tsx`), `signup/` (`ReferralSourceSelector.tsx`), `create-account/`, `forgot-password/`, `reset-password/`, `verify-email/` (+ `Verify.tsx`), `waiting-on-verification/` (+ `RequestNewVerificationEmail.tsx`), `error/` (+ `AuthErrorContent.tsx`), `impersonate/` (admin-impersonate-user flow), `join/` (a separate, simpler "join the waitlist"-style page — not to be confused with `/join/[token]`, the Join Links redemption page, which lives directly under `src/app/join/`, one level up from `auth/`), `logout/route.ts` (a Route Handler, not a page).

Shared building blocks (this is the "share components/schemas" answer): `src/lib/auth/components.tsx` exports `AuthenticationShell` (session-expiry modal wrapper, used by pages that need to detect a mid-session logout), `SignInButton` (renders the SSO/OAuth button — Google/OIDC/ SAML label variants — with optional reCAPTCHA interception before navigating to the backend `authorizeUrl`), `PasswordRequirements` (the live password-policy checklist, reused verbatim by both `EmailPasswordForm` on `/auth/login` + `/auth/signup` and `JoinLinkForm` on `/join/[token]` — see §7.5), and `EmailPasswordForm` (the actual login/signup Formik form, parameterized by a `label: AuthSubmitLabel` prop — `"submit"` for login, other values for signup/join — driving both the button text and whether the signup-only fields/validation apply). Password-rule primitives (`passwordHasUppercase`, `passwordHasLowercase`, `passwordHasDigit`, `passwordHasSpecialChar`, `passwordMeetsLengthRequirements`) live in `src/lib/auth/utils.ts` and are imported independently by every form that needs them (login form, signup form, join-link form all build their own `Yup` schema from the same primitives plus the server-supplied `authTypeMetadata` policy — there's no single shared `Yup` schema object, but there is a single shared `rule set`).

Important accuracy note on removed SSO surface: `AuthType` (`src/lib/auth/types.ts`) still enumerates `GOOGLE_OAUTH` / `OIDC` / `SAML` / `CLOUD` alongside `BASIC`, and `SignInButton` / `LoginPage.tsx` still contain live branches for rendering "Continue with Google"/"Continue with OIDC SSO"/"Continue with SAML SSO" buttons — so at the component-code level, SSO login UI is not fully deleted. However, directly checking the route tree shows `src/app/auth/oauth/callback/`, `src/app/auth/oidc/callback/`, and `src/app/auth/saml/callback/` are **empty directories with no `page.tsx`** — the actual callback routes that would complete any of those OAuth/SAML handshakes have been removed. In practice, this means those `SignInButton` branches are unreachable dead code paths in this fork (they'd only render if `authTypeMetadata.authType` were one of those values, which no live callback route supports completing) — **Basic (email/password) auth and Join Links are the only functional ways to sign in** in this build, despite the SSO-flavored code still sitting in the tree. This is consistent with, but more nuanced than, a flat "SSO was removed" — it's removed at the routing layer, not fully scrubbed from the component layer. The connector-side and Craft-external-app-side OAuth callback pages (`src/app/admin/connectors/[connector]/oauth/callback/`, `src/app/federated/oauth/callback/`, `src/app/craft/v1/apps/oauth/callback/`) are a *different* concern (authorizing a data connector or external app, not logging into Orbyte itself) and are all still present and wired up.

Also confirmed absent, matching the brief:

- **Voice mode:** no `voice/` / `waveform/` / `microphone/` / `tts` files or `useVoice` hook anywhere in `web/src`.
 - **MCP server management UI:** `src/hooks/useMcpServers.ts` is a deliberate stub — its own doc comment states "MCP (Model Context Protocol) server support was removed from the backend. This stub keeps the return shape stable for existing consumers... without hitting a route that no longer exists," and it unconditionally returns an empty/null result. Consumers (`AgentsFilters.tsx`, `AgentViewerModal.tsx`, `ChatPreferencesPage.tsx`, `AgentEditorPage.tsx`) still call it defensively, but there is no admin page to register/configure an MCP server, and no admin route resembling `/admin/mcp*` exists.
 - **OpenAPI custom actions/tools UI:** no admin route for creating/managing custom OpenAPI-backed tools exists (confirmed by directory search); `AdminSidebar.tsx` explicitly comments this out as "disconnected from UI." `src/lib/tools/openApiService.ts` exists in the tree but has zero importers anywhere in `src/` — fully dead code.
 - **Image Generation UI:** the `IMAGE_GENERATION_TOOL_ID` / `IMAGE_GENERATION_TOOL_NAME` constants and an `ImageToolRenderer.tsx` message renderer still exist (so a message *timeline packet* of that type would still render if one ever arrived), but `AgentEditorPage.tsx` and `lib/agents/` never reference `IMAGE_GENERATION_TOOL_ID` — there is no way for a user to toggle image generation on for an agent in the current UI. Effectively dead/vestigial rendering support with no reachable entry point.
-

8. Existing Frontend "APIs" Reference (key `svc.ts` /hook files by feature)

File	Purpose
<code>web/src/lib/users/svc.ts</code>	Login (<code>basicLogin</code>), signup (<code>basicSignup</code>), logout, <code>getCurrentUser</code> , token refresh, user personalization/default-model PATCHers, display-name/initials helpers.
<code>web/src/lib/users/svcSS.ts</code>	Server-side (cookie-forwarding) current-user lookup, used from layouts/pages that need to redirect before any client JS runs.
<code>web/src/lib/auth/svc.ts</code> / <code>svcSS.ts</code>	Email verification requests, OAuth captcha verification, server-side auth-requirement checks (<code>requireAuth</code> , <code>requireAdminAuth</code>).
<code>web/src/lib/auth/hooks.ts</code>	<code>useAuthTypeMetadata</code> , <code>useTokenRefresh</code> , <code>useSessionWatcher</code> (detects server-side session expiry mid-app-use).
<code>web/src/lib/auth/components.tsx</code>	Shared auth UI: <code>SignInButton</code> , <code>EmailPasswordForm</code> , <code>PasswordRequirements</code> , <code>AuthenticationShell</code> .
<code>web/src/views/admin/GroupsPage/svc.ts</code>	Create/rename/update/delete user groups; agent- and doc-set-group sharing diffs; group token-rate-limit CRUD.
<code>web/src/views/admin/GroupsPage/joinLinkSvc.ts</code>	Create and revoke group join links.
<code>web/src/views/admin/GroupsPage/useGroupMemberCandidates.ts</code>	Curator-safe (non-admin-only) fetch of user + service-account candidates for the group member picker.
<code>web/src/lib/agents/svc.ts</code> / <code>hooks.ts</code>	Agent (persona) CRUD, pinning, sharing, labels; <code>useAgents</code> , <code>useAgent</code> , <code>useLabels</code> , <code>useMcpServersForPersonaEditor</code> .
<code>web/src/lib/chat/svc.ts</code>	Chat session backend calls (send message plumbing feeds through <code>app/app/services/*</code> , but session-level ops live here).
<code>web/src/lib/settings/hooks.ts</code>	<code>useSettings()</code> — the single merged settings+enterprise-settings hook; <code>useIsSearchModeAvailable()</code> .
<code>web/src/providers/UserProvider.tsx</code>	<code>useUser()</code> — current user, role flags (<code>isAdmin</code> / <code>isCurator</code>), and every user-preference mutator (theme, chat background, auto-scroll, pinned agents, default model, etc.).
<code>web/src/lib/connectors/*</code> , <code>web/src/hooks/useCCPairs.ts</code>	Connector/CC-pair listing and status.
<code>web/src/lib/tools/{fetchTools,mcpService,mcpUtils,interfaces}.ts</code>	Tool metadata for chat/agent tool toggles (MCP-related calls are effectively neutered per \$7.9).
<code>web/src/lib/build/*</code> , <code>web/src/app/craft/services/*</code>	Craft (build) session/file/artifact/scheduled-task backend calls.
<code>web/src/lib/billing/*</code> , <code>web/src/hooks/useCloudSubscription.ts</code> , <code>useLicense.ts</code>	Tier/billing/license state (<code>Tier</code> enum, <code>tierAtLeast()</code> gating used throughout Groups/agents).
<code>web/src/hooks/useNotifications.ts</code>	In-app notification center (bell icon) fetch/pagination.
<code>web/src/lib/fetcher.ts</code>	Not a feature API, but the load-bearing shared primitive every hook above builds on: <code>errorHandlingFetcher</code> , <code>skipRetryOnAuthError</code> , <code>FetchError</code> / <code>RedirectError</code> .
<code>web/src/lib/swr-keys.ts</code>	Not an API caller itself, but the map every file above uses to know <i>which</i> URL/key to hit — start here to find the right endpoint for any feature.

9. Testing

Playwright E2E: `web/tests/e2e/`, run via `bunx playwright test <name>` (per root `AGENTS.md`) or the `playwright / test` scripts in `package.json`. Structure:

- `pages/` — Page Object Model classes (one per UI surface, e.g. `AgentEditorPage.ts`, `AdminSecurityPage.ts`, `ChatPreferencesPage.ts`). Nested composite objects expose sub-objects (e.g. a chat page object exposes `.inputBar`).
- Spec files grouped by feature directory: `admin/` (includes `admin/groups/groups.spec.ts` + `admin/groups/GroupsAdminPage.ts` + `admin/groups/fixtures.ts` — direct coverage of the Groups feature), `agents/`, `auth/`, `chat/`, `connectors/`, `craft/`, `onboarding/`, `scheduled-tasks/`, `settings/`.
- `fixtures/` — shared Playwright fixtures (`eeFeatures.ts` for EE-gated test setup, `llmProvider.ts`, sample binary files for upload tests).
- `utils/` — non-POM helpers: `auth.ts`, `chatMock.ts` / `chatStream.ts` (mocking/asserting SSE chat streams), `agentUtils.ts`, `dragUtils.ts`, `visualRegression.ts`.
- `global-setup.ts` — creates the seed admin user (per root `AGENTS.md`: `admin_user@example.com` / `TestPassword123!`, "the admin user created by the playwright global setup").
- `constants.ts` — shared test constants (credentials, timeouts, etc.).
- Hard rules** are documented in `web/tests/e2e/README.md`: always use the Page Object Model (specs never construct raw locators), locator priority is `data-testid` / `aria-label` → role-based → text → CSS-as-last-resort, and always use Playwright's auto-retrying `expect(locator).*` matchers instead of one-shot reads like `getAttribute()` / `evaluate()` / `textContent()` for anything that can change asynchronously (React state updates, effects, streaming).

Jest / component tests: co-located `*.test.ts` / `*.test.tsx` files sit next to the source they cover throughout `src/` (e.g. `src/app/app/services/packetUtils.test.ts`, `src/hooks/__tests__/`, `src/lib/__tests__/`, `src/providers/__tests__/`, `src/views/__tests__/`, Craft's own `*.test.tsx` files beside its components/hooks). Run via `bun test` / `npm test` → `jest` (with `test:watch`, `test:coverage`, `test:ci` — `--maxWorkers=2 --silent --bail` for CI, `test:changed` / `test:diff` for local iteration). Uses `@swc/jest` for fast TS transform and `jest-environment-jsdom` for DOM APIs; a separate `web/tests/README.md` (not read in this pass) documents Jest/RTL conventions specifically.

PART TWO

Backend Architecture

The FastAPI service, database layer, auth & permissions, the RAG/search/indexing pipeline with a deep algorithmic dive, chat & agent orchestration, the Craft sandbox, background workers, and a full REST API reference.

Orbyte is a rebranded, trimmed-down fork of Onyx (formerly Danswer). This document is grounded entirely in the code at `backend/` as it exists today. Where upstream Onyx had a feature that this fork removed (Voice, Image Generation, MCP tool integration, OpenAPI custom actions, SSO/OIDC/SAML/SCIM login, multi-tenant Stripe billing), this doc says so explicitly rather than describing it as present. Two subsystems — **Groups/Curators** and **Join Links** — are call-outs because they are *new*, custom-built for this deployment, not upstream Onyx.

1. Tech Stack

Pulled from `backend/requirements/{default,ee,model_server}.txt` (uv-exported lockfiles) and actual imports in the code — not guessed.

Core web/API

- **FastAPI** (`fastapi==0.133.1`) — the API server framework (`onyx/main.py` , `ee/onyx/main.py`).
- **Uvicorn** — ASGI server that runs the FastAPI app (`uvicorn.run(app, ...)` in `onyx/main.py`).
- **Starlette** — underlies FastAPI (middleware pipeline: CORS, captcha, client-IP, latency logging, Prometheus).
- **Pydantic v2** (`pydantic==2.11.7` , `pydantic-settings`) — request/response schemas and most internal data models (`onyx/*/models.py`).
- **fastapi-users** (`fastapi-users==15.0.4` , `fastapi-users-db-sqlalchemy`) — the entire authentication system: registration, login, password reset, email verification, user CRUD. See §5.
- **fastapi-limiter** — rate limiting for auth endpoints.

- **prometheus-fastapi-instrumentator** — `/metrics` endpoint and HTTP latency histograms.
- **sentry-sdk** — error tracking, wired into both FastAPI and Celery.

Database / persistence

- **PostgreSQL** — the system of record. Accessed via **SQLAlchemy 2.0** (`sqlalchemy==2.0.50`) with both a sync engine (`onyx/db/engine/sql_engine.py`) and an async engine (`onyx/db/engine/async_sql_engine.py`).
- **Alembic** (`alembic==1.18.4`) — schema migrations. Two independent migration trees exist: `backend/alembic/` (398+ versioned migration files, the main public/tenant schema) and `backend/alembic_tenants/` (a separate tree for the multi-tenant registry — `public.tenant` bookkeeping tables that exist once per Postgres instance, not per tenant schema).
- **asyncpg / psycopg2-binary** — the actual PostgreSQL drivers underneath the async/sync engines.
- **pgvector-less design**: vector search is NOT done in Postgres — see Document Index below.

Search / vector index

- **OpenSearch** (`opensearch-py==3.2.0`) — the current/target document index backend (`onyx/document_index/opensearch/`), used for hybrid keyword+vector search, replacing Vespa.
- **Vespa** — the original document index backend (`onyx/document_index/vespa/`), still present and dual-run during migration (see §4/§6). Selected via `ONYX_DISABLE_VESPA` / `ENABLE_OPENSEARCH_INDEXING_FOR_ONYX` flags in `onyx/document_index/factory.py`.
- **chonkie** (`chonkie==1.0.10`) — the sentence-aware chunking library that backs `onyx/indexing/chunker.py`.
- **sentence-transformers** (model-server only) — runs the actual embedding + cross-encoder reranking models locally.
- **nltk, tiktoken, tokenizers** — tokenization/stopword utilities used in preprocessing and chunk sizing.

Object storage

- **boto3 / aioboto3 / aiobotocore** — S3-compatible client, used against either real AWS S3 or a self-hosted **MinIO** instance (S3 API compatible) for the `FileStore` (`onyx/file_store/s3_key_utils.py`, `gcs_file_store.py`). A Postgres-backed `FileStore` (`postgres_file_store.py`) and a GCS backend also exist — selectable via `FILE_STORE_BACKEND`.

Background processing

- **Celery** (`celery==5.5.1`) + **kombu/billiard** — the task queue framework driving all asynchronous work (indexing, permission sync, pruning, Craft sandbox lifecycle, scheduled tasks). See §9.
- **Redis** (`redis==5.0.8`) — Celery's broker/result backend, plus distributed locks (`redis.lock.Lock`), rate limiting, and the general-purpose cache backend (`onyx/cache/redis_backend.py`). A Postgres-backed cache backend also exists for the no-Celery/no-vector-DB deployment mode (`onyx/cache/postgres_backend.py`).
- **supervisor** (`supervisord.conf`) — process manager used in the combined-container deployment to run api-server + multiple Celery workers + beat under one entrypoint.

LLM layer

- **LiteLLM** (`litellm==1.89.4`) — the multi-provider LLM abstraction underneath `onyx/llm/multi_llm.py`'s `LiteLLMLLM`, giving one call surface across Anthropic, OpenAI, Google Vertex/Gemini, Azure, Bedrock, Ollama, OpenRouter, and other OpenAI-compatible endpoints.
- **openai, google-genai, google-cloud-aiplatform, cohere, voyageai** — official SDKs used for particular providers/embeddings/reranking where LiteLLM delegates to them.
- **tiktoken** — token counting for context-window budgeting (`onyx/llm/utils.py`).

Observability / tracing

- **braintrust, langfuse, openinference-instrumentation** — pluggable LLM tracing backends (`onyx/tracing/braintrust_tracing_processor.py`, `langfuse_tracing_processor.py`). See §12.
- **opentelemetry-*** — OTLP export plumbing used by the tracing framework.
- **sentry-sdk[celery,fastapi]** — exception monitoring across both the API server and Celery workers.

Connectors / document processing

- **unstructured, markitdown, pypdf/pypdfium2, python-docx, python-pptx, openpyxl/xlrd/xlsxwriter, beautifulsoup4/lxml/trafilatura** — file/HTML parsing for the many connectors (`onyx/connectors/*`).

- **playwright** — headless browser automation used by the web connector / URL crawler and by `OpenURLTool`.
- Per-source SDKs: `atlassian-python-api` (Confluence/Jira), `slack-sdk`, `google-api-python-client` (Drive/Gmail), `office365-rest-python-client` / `msal` (SharePoint/Teams), `simple-salesforce`, `pygithub`, `python-gitlab`, `asana`, `zulip`, `discord-py`, `dropbox`, `hubspot-api-client`, `pyairtable`, etc.

Craft (code-execution) sandbox

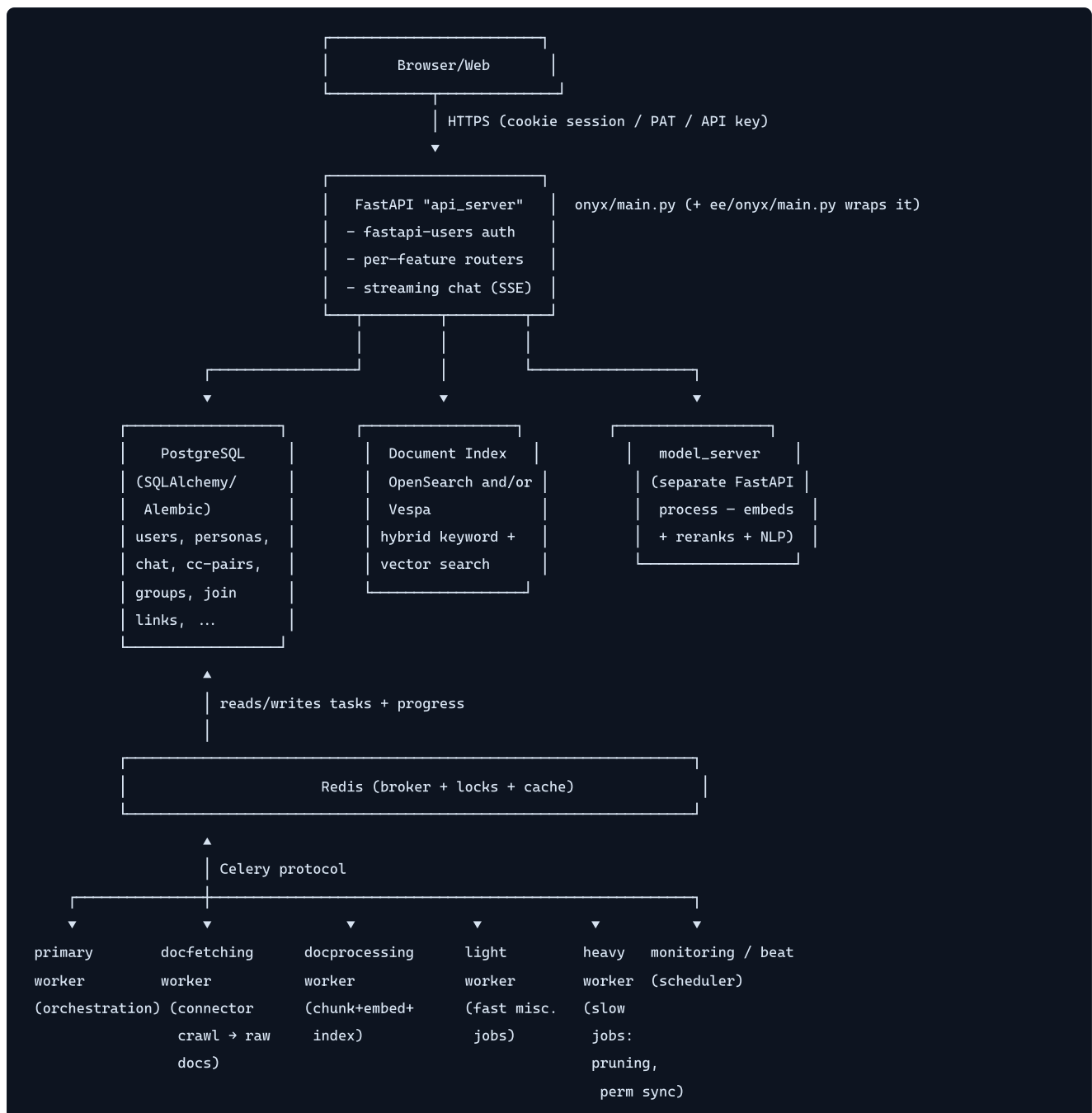
- **docker** (python SDK) and the **Kubernetes API** (`kubernetes==31.0.0` client) — two selectable sandbox backends. See §8.
- **OpenCode** (an external coding-agent CLI baked into the sandbox image) is what actually runs inside the sandbox container/pod; the backend only orchestrates its lifecycle.

Packaging / tooling

- **uv** — the Python package/dependency manager for this repo (`uv.lock` at `backend/`, `requirements/*.txt` are `uv export` outputs, not hand-written).
- **pytest** (`pytest.ini`) — the test runner, with unit/integration/daily/external-dependency test tiers under `backend/tests/`.

2. High-Level Architecture

Orbyte is a multi-process system built around one Postgres database, one document index (OpenSearch and/or Vespa), Redis, S3-compatible object storage, and a fleet of stateless workers.



Request lifecycle (simple chat example)

1. Browser sends `POST /chat/send-chat-message` with a session cookie (fastapi-users auth cookie) or a Personal Access Token.
2. FastAPI dependency chain resolves the `User` (`onyx/auth/users.py`), checks permissions (`onyx/auth/permissions.py`), and loads the `Persona` /agent config.
3. `onyx/chat/process_message.py` builds message history, resolves an `LLM` instance for the persona's configured provider (`onyx/llm/factory.py`), and enters the tool-calling loop (`onyx/chat/llm_loop.py`).
4. If the persona has the internal search tool, the loop calls `onyx/context/search/pipeline.py`, which builds ACL/knowledge-scope filters and queries the **document index** (OpenSearch hybrid query, or Vespa) — this is a synchronous HTTP call from the API server process straight to the search engine, not routed through Celery.
5. The LLM streams tokens back through `onyx/chat/citation_processor.py`, which detects `[1]` / `[[1]]`-style citations and emits `CitationInfo` events alongside the text.
6. Results stream to the browser as newline-delimited JSON ("Packets") over the same HTTP response (`text/event-stream`-flavored streaming), not WebSockets.
7. `onyx/chat/save_chat.py` persists the `ChatMessage` / `ChatSession` rows once the turn completes.

Request lifecycle (document indexing, asynchronous)

1. An admin creates a `Connector` + `Credential` → a `ConnectorCredentialPair` (cc-pair) via `onyx/server/documents/connector.py` / `credential.py` / `cc_pair.py`.
2. Celery Beat's `check-for-indexing` periodic task (or a manual "run once") triggers `onyx.background.celery.tasks.docfetching`, which is picked up by the **docfetching** worker: it invokes the connector (`onyx/connectors/<source>/connector.py`) to pull raw documents/deltas and writes them to intermediate storage.
3. The **docprocessing** worker consumes those raw documents, runs `onyx/indexing/indexing_pipeline.py`: text extraction → chunking (`onyx/indexing/chunker.py`, `chunking/`) → optional contextual-RAG chunk/document summarization via an LLM → embedding (`onyx/indexing/embedder.py` → `model_server`) → writes chunks into OpenSearch/Vespa (`onyx/document_index/*`) and updates Postgres (`Document`, `IndexAttempt`, etc.).
4. Permission sync (ACL propagation) and pruning are separate periodic Celery flows (see §9) that keep the document index's access-control fields in sync with the source system.

Model server

A second, independent FastAPI process (`backend/model_server/main.py`) hosts CPU/GPU-bound NLP work — embeddings (`sentence-transformers`), cross-encoder reranking, and small classification models — so the main API server and Celery workers never load ML models in-process. The API server and workers call it over HTTP (`INDEXING_MODEL_SERVER_HOST/PORT` for indexing-time embedding calls, `MODEL_SERVER_HOST/PORT` for query-time embedding/rerank calls).

3. Directory Structure

`backend/onyx/` (Community edition — the base of the product)

Module	Role	Key files
<code>auth/</code>	fastapi-users wiring, permissions, Join Links, API keys, PATs, JWT/captcha	<code>users.py</code> (UserManager, auth backends), <code>permissions.py</code> (RBAC), <code>join_link.py</code> (token gen/hash), <code>schemas.py</code> (<code>UserRole</code> enum)
<code>db/</code>	All SQLAlchemy models + per-entity CRUD helper modules	<code>models.py</code> (~150 tables), <code>enums.py</code> , <code>user_group.py</code> , <code>join_link.py</code> , <code>persona.py</code> , <code>chat.py</code> , <code>connector_credential_pair.py</code>
<code>db/engine/</code>	Sync + async SQLAlchemy engine construction, tenant-scoped sessions	<code>sql_engine.py</code> , <code>async_sql_engine.py</code> , <code>connection_warmup.py</code>
<code>server/</code>	FastAPI routers, one subpackage per feature area	see §10/§11
<code>chat/</code>	The chat/agent orchestration runtime	<code>process_message.py</code> (entrypoint), <code>llm_loop.py</code> (tool-calling loop), <code>citation_processor.py</code> , <code>compression.py</code> (history compression), <code>save_chat.py</code>
<code>deep_research/</code>	Multi-cycle "Deep Research" agent flow	<code>dr_loop.py</code> (orchestrator), <code>dr_mock_tools.py</code> (clarification/research-agent tool schemas), <code>utils.py</code>
<code>context/search/</code>	The retrieval/search pipeline (distinct from <code>tools/tool_implementations/search</code> , which is the LLM-facing tool wrapper)	<code>pipeline.py</code> , <code>retrieval/search_runner.py</code> , <code>preprocessing/access_filters.py</code>
<code>document_index/</code>	Abstraction over the vector/keyword store; two concrete backends	<code>factory.py</code> (picks OpenSearch vs Vespa), <code>opensearch/</code> , <code>vespa/</code> , <code>interfaces_new.py</code>
<code>indexing/</code>	The document → chunk → embedding pipeline	<code>chunker.py</code> , <code>chunking/</code> (DocumentChunker), <code>embedder.py</code> , <code>indexing_pipeline.py</code>
<code>connectors/</code>	~50 source connectors (Slack, Google Drive, Confluence, Jira, GitHub, Gmail, SharePoint, Salesforce, Notion, Zendesk-like sources, web crawler, etc.)	<code>factory.py</code> + <code>registry.py</code> (lazy <code>DocumentSource</code> → <code>module/class</code> map), <code>interfaces.py</code> (connector contract)

Module	Role	Key files
<code>federated_connectors/</code>	"Federated" (live, non-indexed) search against a source at query time — currently Slack	<code>federated_retrieval.py</code> , <code>slack/</code>
<code>llm/</code>	The multi-provider LLM abstraction	<code>interfaces.py</code> (<code>LLM</code> ABC), <code>multi_llm.py</code> (<code>LitellmLLM</code>), <code>factory.py</code> (persona → LLM resolution), <code>model_capabilities.py</code> , <code>well_known_providers/</code>
<code>tools/</code>	Built-in agent tools + the tool-calling runner	<code>built_in_tools.py</code> (registry), <code>tool_implementations/</code> (search, web_search, open_url, file_reader, memory, coding_agent, bash, python, knowledge_graph), <code>tool_runner.py</code>
<code>natural_language_processing/</code>	Tokenizers, stopwords, the <code>EmbeddingModel</code> / <code>RerankingModel</code> client wrappers that call the model server	<code>search_nlp_models.py</code> , <code>utils.py</code>
<code>background/celery/</code>	All Celery app/worker/task definitions	<code>apps/</code> (per-worker Celery app configs), <code>tasks/</code> (task implementations), <code>configs/</code> (queue/concurrency settings), <code>tasks/beat_schedule.py</code> (periodic schedule)
<code>background/indexing/</code>	Non-Celery indexing helpers shared by tasks	—
<code>file_store/</code>	Object storage abstraction (S3/MinIO, GCS, Postgres-blob)	<code>file_store.py</code> , <code>s3_key_utils.py</code> , <code>gcs_file_store.py</code> , <code>postgres_file_store.py</code>
<code>redis/</code>	Redis-backed coordination primitives per connector/cc-pair (distributed locks, per-cc-pair fence keys)	<code>redis_connector*.py</code> , <code>redis_pool.py</code>
<code>cache/</code>	Generic cache abstraction (Redis or Postgres-backed) used outside the Celery broker	<code>factory.py</code> , <code>redis_backend.py</code> , <code>postgres_backend.py</code>
<code>error_handling/</code>	The single structured-exception convention	<code>exceptions.py</code> (<code>OnyxError</code>), <code>error_codes.py</code> (<code>OnyxErrorCode</code> enum: code string + HTTP status)
<code>tracing/</code>	LLM call tagging + pluggable tracing backends	<code>flows.py</code> (<code>LLMFlow</code> tag enum), <code>braintrust_tracing_processor.py</code> , <code>langfuse_tracing_processor.py</code> , <code>setup.py</code>
<code>prompts/</code>	All system/task prompt text, organized by feature (deep_research, chat, contextual RAG, etc.)	—
<code>sandbox_proxy/</code>	Backend-side proxy that lets a Craft sandbox reach the outside world under policy	—
<code>hooks/</code>	A lightweight extension-point registry (<code>HookPoint</code> : document ingestion / push / query processing) with <code>HARD</code> / <code>SOFT</code> fail strategy	<code>registry.py</code>
<code>kg/</code>	Knowledge-graph extraction (entities/relationships derived from documents)	works with <code>db/entities.py</code> , <code>db/relationships.py</code> , <code>KGEntity</code> / <code>KGRelationship*</code> models
<code>feature_flags/</code>	Feature-flag plumbing (PostHog-backed toggles referenced elsewhere, e.g. Craft gating)	—
<code>evals/</code>	Offline evaluation harness for search/answer quality	—

Module	Role	Key files
<code>skills/</code>	"Skills" — reusable packaged instructions pushed into Craft sandboxes, and shareable Skill objects (<code>db/skill.py</code>)	<code>builtin/</code>
<code>code_interpreter/</code> (via <code>db/code_interpreter.py</code> , <code>server/manage/code_interpreter/</code>)	Config for an external code-execution/interpreter server used outside of Craft	—
<code>coding_agent/</code>	Support code for the <code>CodingAgentTool</code> (drives the same OpenCode-style coding agent as Craft, but as an in-chat tool)	—
<code>external_apps/</code>	OAuth-connected "external apps" (Google Calendar/Drive, Gmail, Slack, Linear, GitHub, HubSpot, Notion, or custom) that Craft/tools can call out to with user-approved scopes	<code>db/external_app.py</code> , <code>db/enums.py::ExternalAppType</code> / <code>EndpointPolicy</code>
<code>onyxbot/</code>	Slack/Discord bot integration surface (distinct from the Slack <i>connector</i> , which only indexes; this is the bot that answers in-channel)	—

`backend/ee/onyx/` (Enterprise-tier code that remains)

The EE package is thin — most files are genuinely still enterprise-tier gated, but a few (billing, SSO OAuth login) are vestigial dead code that exists but is never wired into the running app.

Module	Status	Notes
<code>main.py</code>	Active	Wraps <code>onyx.main.get_application()</code> , adds <code>tier_gate</code> + <code>license_enforcement</code> (self-hosted) or <code>tenant_tracking</code> (multi-tenant) middleware, registers EE-only routers.
<code>server/user_group/</code>	Active, Business-tier gated	The <i>original</i> upstream-Onyx "User Groups" feature: ties a <code>UserGroup</code> to document-level ACLs — connector-credential-pairs, document sets, personas, LLM providers, token rate limits — and drives permission-sync-to-index. See §5 for how this differs from the new CE Groups/Curators system.
<code>server/analytics/</code> , <code>server/query_history/</code> , <code>server/reporting/</code>	Active, Business/Enterprise-tier gated	Query/user/persona analytics, full chat-session history browsing/export, usage-report CSV export.
<code>server/manage/standard_answer.py</code>	Active, Enterprise-tier gated	Canned "standard answer" auto-responses for Slack/onyxbot.
<code>server/oauth/</code>	Active	<code>google_drive.py</code> , <code>confluence_cloud.py</code> , <code>slack.py</code> — OAuth for connector credential acquisition (letting a connector authenticate to Google Drive/Confluence/Slack), not user login/SSO.
<code>server/enterprise_settings/</code>	Active, Enterprise-tier gated for writes	Reduced to whitelabel <code>application_name</code> branding fallback only — logo upload/custom-analytics-script/admin UI removed.
<code>server/evals/</code>	Active, Enterprise-tier gated	Server-triggered eval runs.
<code>server/documents/cc_pair.py</code> (ee)	Active	Adds <code>/sync-permissions</code> and <code>/sync-groups</code> admin endpoints on top of the CE cc-pair router.
<code>server/token_rate_limits/</code>	Active, Enterprise-tier gated	Per-user-group / per-user token rate limits.

Module	Status	Notes
<code>server/middleware/tier_gate.py</code> , <code>license_enforcement.py</code>	Active	The two gating mechanisms — see §3 tier table below.
<code>server/billing/</code> , <code>server/tenants/billing_api.py</code> , <code>server/tenants/proxy.py</code> (Stripe bits)	Dead code, not routed	Files still exist (<code>stripe==10.12.0</code> is still a dependency) but <code>ee/onyx/main.py</code> never calls <code>include_router</code> for <code>billing_router</code> ; there is no live Stripe integration in this deployment. Confirms the task brief: billing/Stripe has been removed from the running product even though some source files remain.
<code>server/tenants/</code> (non-billing: <code>admin_api.py</code> , <code>anonymous_users_api.py</code> , <code>team_membership_api.py</code> , <code>tenant_management_api.py</code> , <code>user_invitations_api.py</code>)	Active only if <code>MULTI_TENANT=true</code>	Multi-tenant provisioning/impersonation/invite endpoints; only registered in <code>ee/onyx/main.py</code> inside <code>if MULTI_TENANT:</code> . Single-tenant self-hosted deployments never mount this router.
<code>db/user_group.py</code> , <code>db/persona.py</code> , <code>db/connector*.py</code> , <code>db/document*.py</code> , <code>db/analytics.py</code> , <code>db/query_history.py</code> , <code>db/license.py</code> , <code>db/standard_answer.py</code> , <code>db/token_limit.py</code> , <code>db/usage_export.py</code>	Active	EE-specific query helpers layered on the same <code>onyx/db/models.py</code> tables.
<code>external_permissions/</code>	Active	Post-query censoring hook for connectors (e.g., Salesforce field-level permissions) invoked from <code>onyx/context/search/pipeline.py</code> via <code>fetch_ee_implementation_or_noop</code> .

How EE/CE swapping works: `onyx/utils/variable_functionality.py` provides `fetch_versioned_implementation` / `fetch_ee_implementation_or_noop` — CE code calls these to optionally invoke an EE-only implementation by dotted path, falling back to a no-op when running the CE-only build (`global_version.is_ee_version()` toggled by `set_is_ee_based_on_env_variable()`). In practice, this fork always ships the `ee/` package alongside `onyx/`, so **the EE code is always present**; what actually differs per deployment is the **tier gate**, not a build flag.

Tier gating today (`ee/onyx/configs/license_enforcement_config.py::PATH_PREFIX_MIN_TIER`, enforced by `ee/onyx/server/middleware/tier_gate.py`):

- `Tier.BUSINESS`: `/admin/chat-sessions`, `/admin/query-history`, `/admin/usage-report`, `/analytics/admin`, `/admin/enterprise-settings`, `/manage/admin/user-group` (the old EE group/ACL feature).
- `Tier.ENTERPRISE`: `/admin/enterprise-settings/custom-analytics-script`, `/admin/enterprise-settings/scim`, `/manage/admin/standard-answer`, `/admin/token-rate-limits`, `/admin/hooks`, `/analytics`, `/evals`, `/scim`.
- Everything else (including the new `/manage/admin/groups` Groups/Curators API and Join Links) is **ungated** — available at every tier.
- `LICENSE_ENFORCEMENT_ENABLED` (default `false`) turns the whole enforcement middleware on/off for self-hosted deployments; multi-tenant/cloud instead uses `is_tenant_gated()` against a control-plane billing check.

4. Database Layer Deep Dive

Core entities (all in `onyx/db/models.py`, one shared `Base` / one Postgres schema per tenant)

- `User` (`SQLAlchemyBaseUserTableUUID` + fastapi-users mixins) — has `role: UserRole` (`limited` / `basic` / `admin` / `curator` / `global_curator` / `slack_user` / `ext_perm_user`), `account_type: AccountType` (`STANDARD` / `BOT` / `EXT_PERM_USER` / `SERVICE_ACCOUNT` / `ANONYMOUS`), `effective_permissions: JSONB` (a cached, expanded set of `Permission` enum strings — see §5), plus UI/chat preference columns (theme, language, default model, pinned assistants, memory settings).
- `UserGroup` — `id`, `name` (unique), `is_up_to_date` (whether the index-side ACL sync has caught up), `is_default` (protects "Basic"/"Admin" seed groups from deletion), relationships to users, cc-pairs, personas, skills, document sets, credentials, MCP servers, and `permission_grants`.

- `User__UserGroup` — the membership association table; carries the crucial `is_curator: bool` column. A row with `is_curator=True` means that user administers *that specific group* (connectors, document sets, membership) without being a global admin.
- `PermissionGrant` — one row per `(group_id, permission)` granted to a `UserGroup`, where `permission` is a `Permission` enum value (see §5). Implied permissions (the `READ_*` /API-surface scopes) are rejected at the model layer via a `@validates` check — they must never be stored directly, only derived at read time.
- `GroupJoinLink` (NEW, this fork) — `token_hash` (SHA-256 hex, unique), `user_group_id`, `created_by`, `is_reusable`, `use_count`, `expires_at`, `revoked_at`. See §5 for the full flow.
- `Persona` — the "assistant"/agent config: prompt, model overrides, attached `Tool`s (via `Persona__Tool`), attached document sets (`Persona__DocumentSet`), user files (`Persona__UserFile`), hierarchy nodes, sharing (`Persona__User`, `Persona__UserGroup`, `public_permission`), labels.
- `DocumentSet` — a named, curated collection of documents (built from cc-pairs) that a persona or a search filter can scope to; `DocumentSet__ConnectorCredentialPair` and `DocumentSet__UserGroup` join tables gate which cc-pairs feed it and which groups can see it.
- `Connector` / `Credential` / `ConnectorCredentialPair` (cc-pair) — a `Connector` is a source *type* + *config* (e.g., "Confluence space X"); a `Credential` is the secret material (OAuth token, API key — encrypted via `EncryptedString` / `EncryptedJson` custom SQLAlchemy types); a cc-pair is the *runtime binding* of the two, carrying `status` (`SCHEDULED` / `INITIAL_INDEXING` / `ACTIVE` / `PAUSED` / `DELETING` / `INVALID`) and `access_type` (`PUBLIC` / `PRIVATE` / `SYNC`).
- `IndexAttempt` — one row per indexing run of a cc-pair, tracks `IndexingStatus` (`not_started` / `in_progress` / `success` / `canceled` / `interrupted` / `failed` / `completed_with_errors`) plus checkpoint/resume state.
- `ChatSession` / `ChatMessage` — the conversation tree (branching supported via parent/child message links), each message optionally carrying `ToolCall` rows and `SearchDoc` citations (`ChatMessage__SearchDoc`).
- `SearchSettings` — the active embedding-model configuration (model name, dimension, `index_name`, provider), with `IndexModelStatus` (`PAST` / `PRESENT` / `FUTURE`) supporting a live re-embedding migration (a "future" search-settings row is built in the background, then swapped to "present").
- `LLMProvider` / `ModelConfiguration` / `LLMModelFlow` — the configured LLM providers (keys, base URLs) and which specific models are enabled per provider, plus per-model-flow overrides (chat vs. vision vs. contextual-RAG vs. reasoning models can differ).
- Knowledge-graph tables (`KGEntity`, `KGRelationship`, `KGEntityType`, plus `*ExtractionStaging` variants) — support the `kg/` module's document-derived entity/relationship graph.
- Craft/Build tables — `BuildSession`, `Sandbox`, `Artifact`, `Snapshot`, `BuildMessage`, `ActionApproval`, `ScheduledTask`, `ScheduledTaskRun` — back the Craft feature (§8).
- `SSOProvider`, `SamlAccount`, `ScimToken` / `ScimUserMapping` / `ScimGroupMapping` tables still exist in the schema (legacy/vestigial from upstream Onyx's SSO support) but this fork's `onyx/main.py` only ever mounts the Basic email/password fastapi-users auth router — see §5.

Alembic migrations

- `backend/alembic/` is the primary migration tree (398+ revision files under `versions/`), applied against each tenant's Postgres **schema** in single-tenant mode (there is effectively one schema, `public` / `POSTGRES_DEFAULT_SCHEMA`).
- `backend/alembic_tenants/` is a **second, independent** Alembic tree for the small set of tables that exist exactly once per Postgres instance regardless of tenant count (tenant directory / provisioning bookkeeping under the `public` schema in multi-tenant mode) — kept separate so per-tenant schema migrations and instance-wide migrations don't collide.
- In multi-tenant mode (`MULTI_TENANT=true`, only meaningfully exercised via the EE tenants code), each tenant gets its own Postgres **schema** (not a separate database), and the primary Alembic tree is run once per schema during provisioning (`ee/onyx/server/tenants/provisioning.py`) — `CURRENT_TENANT_ID_CONTEXTVAR` / `shared_configs/contextvars.py` is how request-scoped code picks the right schema at runtime (`onyx/db/engine/sql_engine.py::get_session_with_current_tenant`).
- In single-tenant mode (the default and the only mode this fork's install docs describe), everything runs in `POSTGRES_DEFAULT_SCHEMA` and the tenant machinery is inert.

5. Auth & Permissions Deep Dive

fastapi-users setup

- `onyx/auth/users.py` builds a `fastapi_users.FastAPIUsers[User, uuid.UUID]` instance around `UserManager(UUIDMixin, BaseUserManager[User, uuid.UUID])`.
- `AUTH_TYPE` is effectively fixed to `basic` in this fork (`onyx/main.py` comment: "Basic (local email/password) auth only — Google OAuth, OIDC, and SAML login have been removed... `AUTH_TYPE` is fixed to BASIC"). Passing `AUTH_TYPE=cloud` raises at startup; `AUTH_TYPE=disabled` is downgraded to a warning and treated as `basic`.
- The registered auth routers (`onyx/main.py`) are only: `fastapi_users.get_auth_router` (login/logout), `get_register_router`, `get_reset_password_router`, `get_verify_router`, `get_users_router`, plus a custom `mobile_auth_router` and a `get_refresh_router`. No OAuth/OIDC/SAML router is ever mounted.
- Session strategy is selectable via `AUTH_BACKEND`: `redis` (default, session token in Redis), `postgres` (`AccessToken` table), or `jwt` (single-tenant, self-hosted only — explicitly rejected for `MULTI_TENANT` or `AUTH_TYPE=cloud`).
- Passwords hashed via `pwdlib` / `argon2-cffi`. `USER_AUTH_SECRET` signs password-reset tokens, email-verification tokens, and captcha cookies; the app refuses to start on a real deployment without one.

RBAC / `UserRole` and `Permission`

- `UserRole` (`onyx/auth/schemas.py`): `LIMITED`, `BASIC`, `ADMIN`, `CURATOR`, `GLOBAL_CURATOR`, `SLACK_USER`, `EXT_PERM_USER`. `ADMIN` is the superuser role; `CURATOR` / `GLOBAL_CURATOR` are legacy upstream role labels still on the `User` row, but the group-scoped curator behavior actually used by this fork's Groups system is driven by the **per-group** `User__UserGroup.is_curator` flag, not this top-level role (a user can be `role=BASIC` and still be a curator of one specific group).
- `Permission` (`onyx/db/enums.py`) is a flat token vocabulary: `basic`, `read:*` (implied-only, never granted directly), `manage:agents`, `add:agents`, `manage:document_sets`, `add:connectors`, `manage:connectors`, `manage:llms`, `manage:user_groups`, `create:user_api_keys`, `create:service_account_api_keys`, `create:slack_discord_bots`, `craft_sandbox`, and the override `admin` (`FULL_ADMIN_PANEL_ACCESS`) which short-circuits every check to "allowed."
- `onyx/auth/permissions.py::resolve_effective_permissions` expands a user's *directly granted* permission set (`PermissionGrant` rows on their groups, aggregated onto `User.effective_permissions` as a cached JSONB list) through a static `IMPLIED_PERMISSIONS` map (e.g., `manage:connectors` implies `add:connectors` implies `read:connectors`). `require_permission(...)` is the FastAPI dependency factory every gated route uses; it also intersects against `request.state.token_scopes` so a scoped Personal Access Token can never exceed what its issuing user could do.
- This is a genuinely custom-built permission system layered on top of the older upstream Onyx `UserRole`-only model — it did not exist in the same form upstream.

Groups / Curators (NEW to this deployment)

Two parallel "group" systems exist in the codebase and it is easy to conflate them:

- `/manage/admin/groups` (`onyx/db/user_group.py`, `onyx/server/features/user_group/api.py`) — the **new**, lightweight, always-available (ungated) group system built for this product. A `UserGroup` here is just: a set of member users, a set of `is_curator` flags, and a set of `PermissionGrant`s. `user_curates_group(db_session, user_id, group_id)` (`onyx/db/user_group.py`) is the core check — it looks for a `User__UserGroup` row matching `(user_id, group_id, is_curator=True)`. Curators (checked via `current_curator_or_admin_user` dependency) can manage membership and Join Links for groups they curate, without needing the global `admin` role.
- `/manage/admin/user-group` (`ee/onyx/db/user_group.py`, `ee/onyx/server/user_group/api.py`) — the **original upstream Onyx Enterprise** "User Groups" feature, operating on the *same* `UserGroup` / `User__UserGroup` tables but additionally wiring groups to connector-credential-pairs, document sets, personas, LLM providers, and token rate limits for **document-level ACL propagation into the search index**. This is Business-tier gated (`PATH_PREFIX_MIN_TIER["/manage/admin/user-group"] = Tier.BUSINESS`). A new engineer should think of these as: "basic RBAC group management" (new, free) vs. "document-permission-sync groups" (old upstream EE feature, paid tier).

Join Links (NEW to this deployment — full flow)

Join Links let an admin/curator onboard new members into a specific group **without sending an email invite** — the possession of an unguessable URL is the invitation.

1. **Creation** (`POST /manage/admin/groups/{group_id}/join-links` , `onyx/server/features/join_link/api.py`): only an admin (any group) or a curator of that specific group (`current_curator_or_admin_user` + `_require_group_access`) may create a link. `onyx/auth/join_link.py::generate_join_link_token()` builds `"onyx_join_" + secrets.token_urlsafe(48)` — a cryptographically random, unguessable token. Only its **SHA-256 hash** (`hash_join_link_token`) is persisted on `GroupJoinLink.token_hash` ; the raw token is returned once in the API response and never stored, so even a full DB dump cannot be used to mint working links. Links can be `is_reusable` or single-use, and can carry an `expires_at` .
2. **Lookup** (`GET /join-link/{token}` , public route): resolves the token, returns `{valid, group_name}` so the frontend can show "You're joining " before the user creates an account. `onyx/db/join_link.py::resolve_join_link` rejects the link if it's revoked, expired, or (for single-use links) already has `use_count ≥ 1` .
3. **Redemption** (`POST /join-link/{token}/redeem` , public route): calls `userManager.create(user_create, safe=True, join_token=token)` . This is the key bypass point: in `onyx/auth/users.py` , the normal registration path calls `verify_email_is_invited(email)` (which checks an admin-configured invite whitelist) **unless a `join_token` was supplied** — i.e., presenting a valid join-link token is treated as equivalent proof of invitation, skipping the email-invite-list check entirely. After the user is created, `add_users_to_user_group(db_session, link.user_group_id, [new_user.id])` adds them to the target group (additive/idempotent — safe even if the group is also the account's default group), and `record_join_link_use__no_commit` increments `use_count` in the same transaction as the group-membership insert.
4. **Revocation** (`DELETE /manage/admin/groups/{group_id}/join-links/{link_id}`): sets `revoked_at` ; `resolve_join_link` treats any non-null `revoked_at` as permanently dead.

This is a genuinely new onboarding primitive — upstream Onyx only supports either open signup or an admin-curated email allowlist; there is no equivalent link-based flow upstream.

6. RAG / Search / Indexing Pipeline — Deep Algorithmic Dive

End-to-end pipeline and the real Celery task/worker names

1. **docfetching** (Celery worker `onyx/background/celery/apps/docfetching.py` , tasks in `onyx/background/celery/tasks/docfetching/tasks.py`) — driven by the Beat-scheduled `check-for-indexing` task (`OnyxCeleryTask.CHECK_FOR_INDEXING` , `onyx/background/celery/tasks/docprocessing/tasks.py::check_for_indexing`) which finds cc-pairs due for a run and kicks off a per-cc-pair fetch. The connector (`onyx/connectors/<source>/connector.py` , resolved lazily via `onyx/connectors/registry.py::CONNECTOR_CLASS_MAP`) pulls raw documents (full or incremental/poll-window based on `IndexingStatus.should_reuse_checkpoint()`), writing intermediate results for the next stage.
2. **docprocessing** (worker `apps/docprocessing.py` , tasks in `tasks/docprocessing/tasks.py`) — runs the actual `onyx/indexing/indexing_pipeline.py` , which:
 - Extracts/normalizes text per document (`onyx/file_processing/`), builds `IndexingDocument` s with `processed_sections` .
 - **Chunks** each document (`onyx/indexing/chunker.py::Chunker` , using `chonkie.SentenceChunker` under the hood — see below).
 - Optionally computes **contextual-RAG** chunk/document summaries via an LLM (`LLMFlow.CONTEXTUAL_RAG_DOC_SUMMARY` / `CONTEXTUAL_RAG_CHUNK_CONTEXT`).
 - **Embeds** chunks (`onyx/indexing/embedder.py::DefaultIndexingEmbedder` , calling the model server).
 - **Indexes** into the document index (`onyx/document_index/factory.py::get_all_document_indices` — writes to Vespa and/or OpenSearch depending on which are enabled) and updates Postgres bookkeeping (`Document` , `IndexAttempt` , `DocumentByConnectorCredentialPair`).
3. **Permission sync** (`check-for-doc-permissions-sync` / `check-for-external-group-sync` , EE-gated by `DOC_PERMISSION_SYNC_ENABLED` -style flags) periodically re-pulls ACLs from the source system and pushes updated `access_control_list` values into the index without re-indexing content.
4. **Pruning** (`check-for-pruning`) periodically diffs the source system against indexed documents and deletes ones no longer present/authorized.
5. Query time never touches Celery: the API server calls the document index directly and synchronously.

Chunking strategy

`onyx/indexing/chunker.py::Chunker` wraps three `chonkie.SentenceChunker` instances (blurb, main chunk, mini-chunk) that count tokens using the model's real tokenizer (`onyx/natural_language_processing/utils.py::BaseTokenizer`), not characters. Per document:

- A short **blurb** (title-sized summary) and a **title prefix** are computed and prepended to each chunk's embedding input, plus a **metadata suffix** (source-specific key/value metadata rendered as natural-language text for the semantic embedding, and a flat value list for keyword search) — capped so metadata can never exceed 25% of a chunk's token budget (`MAX_METADATA_PERCENTAGE`).
- **Contextual RAG**: when `USE_CHUNK_SUMMARY` and/or `USE_DOCUMENT_SUMMARY` are enabled, a reserved token budget (`DEFAULT_CONTEXTUAL_RAG_RESERVED_TOKENS = MAX_CONTEXT_TOKENS * (has_chunk_summary + has_doc_summary)`) is carved out of each chunk so an LLM-generated contextual blurb can be embedded alongside the raw chunk text (this is the "prepend an LLM-written 1-2 sentence context to each chunk before embedding" technique — improves retrieval recall for chunks that are ambiguous out of context). Skipped automatically if a whole document already fits in one chunk.
- **Multipass / mini-chunks**: when `enable_multipass` is set, a second, smaller-granularity split (`mini_chunk_splitter`) is embedded per chunk for finer-grained vector matching, in addition to the full chunk embedding.
- **Large chunks**: `generate_large_chunks` optionally merges groups of `LARGE_CHUNK_RATIO` consecutive chunks into a bigger "section" chunk (`large_chunk_reference_ids` links back to the originals) — a second index entry used to catch matches that need more surrounding context than a single small chunk provides.
- No chunk overlap is used by design (`CHUNK_OVERLAP = 0`) — the code comment explains the team found overlaps didn't reliably help quality and complicated clean chunk boundaries.

Embedding model

- **Default model**: `nomic-ai/nomic-embed-text-v1` (`onyx/configs/model_configs.py::DEFAULT_DOCUMENT_ENCODER_MODEL`), 768-dimensional, run **locally** by the model server via `sentence-transformers.SentenceTransformer` (`model_server/encoders.py::get_embedding_model`) — not a hosted API call by default. Alternate pre-configured local options exist in `onyx/configs/embedding_configs.py` : `intfloat/e5-base-v2` , `intfloat/e5-small-v2` , `intfloat/multilingual-e5-base` , `intfloat/multilingual-e5-small` . Cloud embedding providers (Cohere, Voyage, OpenAI, Azure) are also supported through `CloudEmbeddingProvider` — in that case the model server is bypassed and the embedding call goes straight to the provider's API.
- The model server caches loaded `SentenceTransformer` instances in a process-global dict, pins `max_seq_length` , and "pre-warms" RoPE positional caches with a dummy long input so the first real request isn't slow.
- Query-time and passage-time embedding calls carry configured `query_prefix` / `passage_prefix` strings (nomic and e5-family models expect instruction prefixes like `"search_query: "` / `"search_document: "`).

Reranking

`onyx/natural_language_processing/search_nlp_models.py::RerankingModel` — for local models, calls the model server's `/encoder/cross-encoder-scores` endpoint (`DEFAULT_CROSS_ENCODER_MODEL_NAME` env-configurable); for API-based rerank providers it calls the provider directly. Reranking is applied after retrieval to re-score/re-order the hybrid search's top candidates using a (slower, more accurate) cross-encoder rather than the bi-encoder used for the initial vector search.

Hybrid search algorithm (OpenSearch backend — `onyx/document_index/opensearch/search.py`)

OpenSearch's **hybrid query** feature runs multiple independent sub-queries and fuses their scores through a **search pipeline** normalization processor:

- Sub-queries depend on `HYBRID_SEARCH_SUBQUERY_CONFIGURATION` . The simpler/current mode (`CONTENT_VECTOR_TITLE_CONTENT_COMBINED_KEYWORD`) runs exactly two clauses: (a) a `knn` vector search against the content embedding, and (b) a combined title+content BM25-style keyword `bool` / `should` query (title matches heavily discounted since titles are already folded into the content text — boost `0.1` / `0.2` vs. content boosts `1.0` / `1.5` for phrase matches). An older three-clause mode also scores the title vector separately.
- Each sub-query independently returns up to `DEFAULT_NUM_HYBRID_SUBQUERY_CANDIDATES` candidates per shard (`pagination_depth`), then a **normalization pipeline** (`get_normalization_pipeline_name_and_config`) rescales each clause's scores — either **min-max** or **z-score** normalization — and combines them via a weighted **arithmetic mean** (weights sum to 1.0; e.g. content-vector `0.5` / combined-keyword `0.5`). This is the actual "combine keyword + vector" algorithm: two independent recall passes, each independently normalized, then linearly blended — not a manually-coded RRF (reciprocal rank fusion), though OpenSearch also supports that as an alternative technique the code deliberately didn't choose here (the comment set explains the tradeoffs considered, e.g. rejecting `minimum_should_match` and `fuzziness AUTO` as hurting recall on semantic-heavy queries).
- All ACL/tag/document-set/time/hierarchy/knowledge-scope filtering happens as **filter clauses** (no scoring impact, fully cacheable) wrapped around the hybrid query, applied identically to every sub-query so filtered documents are excluded before fusion, not after.
- Vespa (the other backend, `onyx/document_index/vespa/`) implements the equivalent hybrid ranking using Vespa's own YQL/ranking-profile mechanism; the two backends can run **simultaneously** during a live cut-over (`get_all_document_indices`).

writes to both; a `db/opensearch_migration.py`-tracked flag decides which one actually serves *reads*), which is why both `onyx/document_index/vespa/` and `onyx/document_index/opensearch/` are fully present in this codebase — this fork is mid-migration off Vespa onto OpenSearch, not simply "using OpenSearch."

Document permissions/ACL propagation into the index

Every chunk carries an `access_control_list` field in the index (`ACCESS_CONTROL_LIST_FIELD_NAME`). At query time, `_get_acl_visibility_filter` builds an OR filter: `public == true` OR `access_control_list` intersects the caller's ACL entries (computed per-request in `onyx/context/search/preprocessing/access_filters.py::build_access_filters_for_user`). ACL values are written at index time from the connector's permission model and kept current by the periodic doc-permission-sync/external-group-sync Celery tasks (§9) rather than being recomputed per query.

Citation processing (grounding answers back to source chunks)

`onyx/chat/citation_processor.py::DynamicCitationProcessor` runs as a streaming token filter alongside the LLM's raw output:

- Maintains a `citation number → SearchDoc` mapping (each numbered source is a resolved chunk/document surfaced during the tool-calling turn).
- Regex-detects citation markers in three forms: `[1]`, `[1, 2, 3]`, and unicode bracket variants (`[1]`, `[1]`), holding back partial matches (e.g. a trailing `"["`) until they resolve or are proven to not be a citation, to avoid ever emitting a broken partial marker to the client.
- Three modes: `HYPERLINK` (replaces `[1]` with `[[1]](url)` markdown and emits `CitationInfo` events for the UI to render a clickable source card — used for the final answer shown to the user), `KEEP_MARKERS` (leaves `[1]` untouched — used mid-flight in the Deep Research loop so citation numbers can be renumbered/collapsed later), `REMOVE` (strips citations entirely — used for channels like Slack/Discord bots where inline citation UI doesn't exist).
- Deduplicates rapid repeat citations to the same document (`recent_cited_documents`, cleared after 5 non-citation characters) so a document cited three times in one sentence doesn't spam three separate `CitationInfo` events.

7. Chat / Agent Orchestration Deep Dive

Persona/agent config resolution

A `Persona` (DB model, user-facing term "Assistant"/"Agent") bundles: a system prompt, an optional LLM override (`provider_name_override` / `model_version_override`), attached tools (`Persona__Tool` → `Tool` rows, which can wrap a built-in tool by name or reference an external app), knowledge scope (attached document sets, user files, hierarchy nodes), and sharing/visibility rules (owner, `public_permission`, per-user/per-group shares via `Persona__User` / `Persona__UserGroup`, each carrying `PersonaSharePermission.EDITOR` / `VIEWER`).

Prompt construction and the LLM abstraction

- `onyx/llm/factory.py::get_llm_for_persona` (approx.) resolves (`LLMProviderModel`, `model_name`) for a chat turn — checking any override, then the persona's configured provider, then the workspace default — and constructs a `LitellmLLM` (`onyx/llm/multi_llm.py`) wrapping the resolved `LLMConfig` (provider, model, temperature, API key/base/version, max input tokens).
- `LLM` (`onyx/llm/interfaces.py`) is the abstract base every backend implements: `invoke( ... )` (single response) and `stream( ... )` (token iterator), both parameterized by `tools`, `tool_choice`, `structured_response_format`, `reasoning_effort`, and an optional `LLMUserIdentity`. A metaclass hook (`__init_subclass__`) auto-wraps every subclass's `invoke` / `stream` with a tracing span (`onyx/llm/tracing_wrap.py`) so **every** LLM call anywhere in the codebase is captured by the tracing backend even if the call site forgot to add an explicit span — see §12.
- LiteLLM is the actual multi-provider transport underneath `LitellmLLM`: Anthropic, OpenAI, Google (Gemini/Vertex), Azure OpenAI, AWS Bedrock, Ollama, OpenRouter, and generic OpenAI-compatible endpoints are all reachable through one call surface; provider-specific quirks (e.g. Ollama's `num_ctx`, OpenRouter's attribution headers) are patched in via small provider-keyed adapters in `onyx/llm/factory.py` / `well_known_providers/`.

Tool-calling loop

`onyx/chat/llm_loop.py::run_llm_loop` is the core loop: it calls `run_llm_step` / `run_llm_step_pkt_generator` (`onyx/chat/llm_step.py`) with the current message history, tools list, and streaming callback; when the model emits tool calls, `onyx/tools/tool_runner.py` dispatches them, appends `ToolCallSimple` results back into history, and loops until the model produces a plain-text final answer (or a stopping tool fires). Built-in tools registered in `onyx/tools/built_in_tools.py`:

- `SearchTool` (`tool_implementations/search/`) — internal document search (drives the hybrid-search pipeline in §6). Citeable.
- `WebSearchTool` (`tool_implementations/web_search/`) — live web search via a configured provider. Citeable.
- `OpenURLTool` (`tool_implementations/open_url/`) — fetches/reads a specific URL the model names. Citeable.
- `FileReaderTool` (`tool_implementations/file_reader/`) — reads user-uploaded / project files.
- `MemoryTool` (`tool_implementations/memory/`) — reads/writes the per-user `Memory` table (long-term chat memory) when `User.use_memories` / `enable_memory_tool` is on.
- `CodingAgentTool` (`tool_implementations/coding_agent/`) — an in-chat tool variant of the same coding-agent machinery Craft uses, letting a chat turn run a sandboxed coding sub-agent.
- Additional non-"built-in-tool-registry" tool implementations exist under `tool_implementations/bash/`, `tool_implementations/python/`, and `tool_implementations/knowledge_graph/` for more specialized/administrative flows.
- No MCP tool integration and no OpenAPI custom-action tools exist in this fork's tool runner — both were removed from upstream.

Streaming response mechanism

Responses stream over a single HTTP response as an ordered sequence of typed "Packets"

(`onyx/server/query_and_chat/streaming_models.py`: `AgentResponseStart/Delta`, `SectionEnd`, `OverallStop`, `CitationInfo`, `DeepResearchPlanStart/Delta`, etc.), written incrementally by `onyx/chat/emitter.py::Emitter` and consumed as newline-delimited JSON by the frontend — not WebSockets, not raw SSE `text/event-stream` framing (the OpenAPI doc for `/chat/send-chat-message` explicitly documents both a JSON `ChatFullResponse` mode and a streamed mode). `onyx/chat/stop_signal_checker.py` lets the client cancel an in-flight turn (`POST /chat/stop-chat-session/{id}`), and `onyx/chat/chat_processing_checker.py` tracks turn-in-progress state so a client can resume/poll a stream after a disconnect (`GET /chat/chat-session/{id}/resume-stream`).

Deep Research (multi-step agentic flow — still present)

`onyx/deep_research/dr_loop.py` implements a distinct, heavier orchestration mode (selected per-turn, not per-persona) with its own prompts (`onyx/prompts/deep_research/orchestration_layer.py`):

1. **Clarification** step (skippable via `SKIP_DEEP_RESEARCH_CLARIFICATION`) — the model may first ask the user a clarifying question before researching.
2. **Orchestrator loop** — up to `MAX_ORCHESTRATOR_CYCLES = 8` (or `4` for reasoning models) cycles alternating between issuing research sub-tasks (dispatched to a "research agent" tool persona built from `SearchTool` / `WebSearchTool` / `OpenURLTool`, run via `onyx.tools.fake_tools.research_agent::run_research_agent_calls`) and an explicit "think" step, tracked with `CitationMode.KEEP_MARKERS` so citation numbers stay stable across cycles for later collapsing.
3. **Forced report generation** after `DEEP_RESEARCH_FORCE_REPORT_SECONDS = 30 * 60` even mid-cycle, so a run can never hang indefinitely.
4. **Final report** step re-renders the accumulated, deduplicated citations in `HYPERLINK` mode (`collapse_citations`-style renumbering) for the user-facing answer, capped at `MAX_FINAL_REPORT_TOKENS = 20000`.

8. Craft (Code Execution / Sandbox) Deep Dive

"Craft" (product name; code paths still say `build`) is Orbyte's sandboxed coding-agent workspace feature, at `onyx/server/features/build/`.

What it is

Each Craft session gives a user an isolated container/pod running **OpenCode** (an external AI coding-agent CLI, not written by Onyx) with: a scaffolded Next.js app (shadcn/ui + Recharts) for building small web apps, a pre-built Python virtualenv for data work, read-only symlinked access to the user's indexed knowledge files, and generated-output directories (`outputs/web`, `outputs/slides`, `outputs/markdown`, `outputs/graphs`).

Sandbox architecture — two selectable backends (`SANDBOX_BACKEND` env var)

- **Kubernetes mode** (default, production/cloud):
`onyx/server/features/build/sandbox/kubernetes/kubernetes_sandbox_manager.py`. One pod per user (`sandbox-<id>`), with a main `sandbox` app container plus a native **restartable init sidecar** (`sidecar`) that handles push/snapshot control-plane work over the pod lifetime — this requires Kubernetes `>= 1.33` (restartable init sidecars, `initContainers[*].restartPolicy: Always`). The `api_server` drives pod lifecycle + `kubectl exec` via the Kubernetes API; snapshots stream through the sidecar straight to the `api_server`-owned `FileStore`, so the sandbox container itself never holds S3/MinIO credentials.
- **Docker mode** (self-hosted `docker compose`, `install.sh --include-craft`):
`onyx/server/features/build/sandbox/docker/docker_sandbox_manager.py`. One container per user on the same host, `api_server/background` mount `/var/run/docker.sock` and drive containers via the Docker Engine API + `docker exec`. Sandboxes join **only** a dedicated `onyx_craft_sandbox` bridge network — Postgres/Redis/MinIO/model-server are unreachable by compose DNS from inside a sandbox. Containers run `--security-opt no-new-privileges --cap-drop ALL user=1000:1000`, no Docker socket, and a fixed env allowlist (`ONYX_PAT`, `ONYX_SERVER_URL`). `SANDBOX_API_SERVER_URL` must be a real public HTTPS URL — compose-internal hostnames don't resolve from the sandbox's isolated bridge.
- `onyx/server/features/build/sandbox/factory.py::get_sandbox_manager()` is a lazy, thread-safe singleton picking the manager class based on `SANDBOX_BACKEND` — the two implementations share a common abstract interface (`sandbox/base.py::SandboxManager`) so the rest of the codebase (session lifecycle, snapshotting) doesn't care which backend is active.

Session / turn lifecycle

- `session/` — `manager.py` (session create/list/get/delete/restore, sandbox status polling), `sandbox_lifecycle.py` (wake/sleep/idle-timeout logic — `SANDBOX_IDLE_TIMEOUT_SECONDS`, default 900s), `naming.py` (auto-generates a session name via `LLMFlow.BUILD_SESSION_NAMING`), `streaming.py` / `messages.py` (turn message send/interrupt), `md_to_docx.py` (export helper).
- `interactive_turns/` — `executor.py` runs one user→agent turn against the live sandbox; `state.py` / `models.py` track turn state; a turn's live event stream is exposed via `GET /sessions/{id}/turns/{turn_id}/events` and `GET /sessions/{id}/turns/active`.
- `approvals/` — some agent actions require human-in-the-loop approval (`ActionApproval` model, `ApprovalDecision`: `APPROVED` / `REJECTED` / `EXPIRED`, `ApprovalDecidedVia`: `USER` / `PRE_APPROVAL` / `SESSION_GRANT`); `GET /approvals/sessions/{id}/live` streams pending approvals, `POST /approvals/{id}/decision` resolves one.
- `scheduled_tasks/` — Craft sessions can be re-run on a schedule (`ScheduledTask` / `ScheduledTaskRun` models, `ScheduledTaskStatus`: `ACTIVE` / `PAUSED`, dispatched by the Celery Beat task `dispatch-due-scheduled-tasks` / cleaned up by `cleanup-stuck-scheduled-runs`). `SessionOrigin.SCHEDULED` sessions are hidden from the normal sidebar.
- `external_apps/` — lets a Craft session (or a chat tool) call out to a user-approved external API (Google Calendar/Drive, Gmail, Slack, Linear, GitHub, HubSpot, Notion, or a custom app) under an `EndpointPolicy` (`ALWAYS` / `ASK` / `DENY` per action, strictest wins).
- `sandbox_proxy/` (top-level `onyx/` module) is the backend-side reverse proxy the sandbox's outbound calls (to `SANDBOX_API_SERVER_URL`, e.g. for MCP/tool calls back into Onyx) route through, so the sandbox never gets a raw, unrestricted path to internal services.
- Snapshots (`sandbox/snapshot_manager.py`) tar-stream a session's workspace into the `FileStore` so a "sleeping" (Kubernetes pod terminated / Docker container idle) session can be woken and its files restored later — this is what backs `POST /sessions/{id}/snapshot` and `POST /sessions/{id}/restore`.

9. Background Workers (Celery) Deep Dive

Worker apps live in `onyx/background/celery/apps/`, one Python module per worker type, each built on a shared `app_base.py` (handles Sentry/tracing setup, Vespa/OpenSearch readiness waits, structured logging, and a `before_task_publish` / `task_prerun` / `task_postrun` instrumentation hook). Per-worker queue/concurrency tuning lives in the parallel `onyx/background/celery/configs/` package.

Worker (<code>apps/*.py</code>)	Purpose
<code>primary</code>	The general orchestration worker — owns most of the "check-for-X" periodic dispatch logic and lighter-weight coordination tasks (connector deletion orchestration, vespa/opensearch sync fan-out, etc.).
<code>docfetching</code>	Runs connectors (<code>onyx/connectors/*</code>) to pull raw documents/deltas from source systems. CPU/network-bound, one task per cc-pair run.

Worker (<code>apps/*.py</code>)	Purpose
<code>docprocessing</code>	Runs the heavy indexing pipeline: chunking, contextual-RAG summarization, embedding, and writing into the document index. The most resource-intensive worker; scaled independently from docfetching.
<code>user_file_processing</code>	Processes user-uploaded files (chat attachments / project files) — a lighter, faster-turnaround variant of the docprocessing pipeline scoped to <code>UserFile</code> rows, not full connectors.
<code>light</code>	Fast, low-risk misc. jobs that shouldn't wait behind heavy indexing work.
<code>heavy</code>	Slow jobs (pruning, permission sync, hierarchy fetching) that are allowed to run long but shouldn't starve the light queue.
<code>monitoring</code>	Runs the periodic health/metrics tasks: <code>monitor-background-processes</code> , <code>monitor-celery-queues</code> , <code>monitor-process-memory</code> , <code>celery-beat-heartbeat</code> .
<code>scheduled_tasks</code>	Drives Craft's user-defined scheduled task runs (<code>dispatch-due-scheduled-tasks</code> , <code>cleanup-stuck-scheduled-runs</code>) — separate from Beat itself, this is the worker that actually <i>executes</i> a due scheduled Craft session.
<code>beat</code>	Not a task-executing worker — runs Celery Beat , the periodic scheduler, which enqueues the tasks listed below onto the appropriate queues on a timer.
<code>client</code>	A thin Celery client app used by the API server process itself to enqueue tasks (e.g., "run this connector now") without pulling in full worker machinery.

Beat schedule (`onyx/background/celery/tasks/beat_schedule.py::get_tasks_to_schedule`)

Confirmed periodic tasks (`OnyxCeleryTask` enum values), each mapped to a cron/interval entry:

- `check-for-user-file-processing` , `check-for-user-file-project-sync` , `check-for-user-file-delete`
- `check-for-indexing` (the main "is any cc-pair due for a crawl" dispatcher)
- `check-for-port` (search-settings/model migration "port" jobs — re-embedding under a new model)
- `check-for-checkpoint-cleanup` , `check-for-index-attempt-cleanup`
- `check-for-connector-deletion`
- `check-for-vespa-sync` (propagates DB-side changes — e.g. a document set edit — into the index)
- `check-for-pruning`
- `check-for-hierarchy-fetching` (rebuilds folder/space/page hierarchy trees for sources like Drive/Confluence/SharePoint)
- `monitor-background-processes`
- `dispatch-due-scheduled-tasks` , `cleanup-stuck-scheduled-runs` (Craft)
- `cleanup-idle-sandboxes` (Craft)
- `check-for-doc-permissions-sync` , `check-for-external-group-sync` (conditionally scheduled, EE permission-sync features)
- `check-for-auto-llm-update` (conditionally scheduled)
- `scheduled-eval-pipeline` (conditionally scheduled, EE evals)
- `migrate-chunks-from-vespa-to-opensearch` (conditionally scheduled — the live Vespa→OpenSearch backfill job)
- Multi-tenant/cloud-only entries prefixed `ONYX_CLOUD_CELERY_TASK_PREFIX` : `monitor-alembic` , `monitor-celery-queues` , `check-available-tenants` , `monitor-celery-pidbox` (a "cloud beat generator" task fans these out per-tenant).

Redis backs both the Celery broker and a set of per-cc-pair/per-connector coordination primitives (`onyx/redis/redis_connector*.py`) — distributed locks and "fence" keys that prevent, e.g., two overlapping indexing runs for the same cc-pair, or a deletion racing an in-flight index.

The no-Celery deployment mode

`DISABLE_VECTOR_DB=true` is a special lightweight mode (validated at startup in `onyx/main.py::validate_no_vector_db_settings`) that removes Celery/Redis/the vector DB entirely, replacing scheduled work with a simple in-process `onyx.background.periodic_poller` thread. It's mutually exclusive with `MULTI_TENANT` and with `ENABLE_CRAFT` (Craft needs the background workers for sandbox lifecycle management), and it also requires `CACHE_BACKEND=postgres` instead of Redis.

10. API Layer / Server Routers

`onyx/server/` follows a strict **one-router-per-feature** convention: nearly every subpackage exposes `router` (or `basic_router` / `admin_router` / `public_router` when a feature needs separate auth tiers) from an `api.py`, and `onyx/main.py::get_application()` wires each one in with `include_router_with_global_prefix_prepended` (which prepends `APP_API_PREFIX`, empty by default). `ee/onyx/main.py` layers additional EE routers on top of the same base app.

Major routers actually registered in `onyx/main.py` (grouped by area), each with a one-line purpose:

- **Auth:** `fastapi-users login/register/reset-password/verify/users routers` (Basic auth only), `mobile_auth_router` (mobile session bootstrap), `captcha_router` (login captcha verification).
- **Chat & query:** `chat_router` (chat sessions + streaming send-message), `query_router` / `admin_query_router` (one-shot search-and-answer API), `search_api_router` (assistant-scoped search), `federated_router` (live Slack federated search config).
- **Documents/connectors:** `connector_router`, `credential_router`, `cc_pair_router`, `document_router`, `targeted_reindex_router`, `standard_oauth_router` (connector OAuth), `onyx_api_router` (external ingestion API).
- **Personas/agents:** `persona_router` / `admin_persona_router` / `agents_router` / `admin_agents_router`, `default_assistant_router`.
- **Knowledge organization:** `document_set_router`, `hierarchy_router`, `projects_router` (user "Projects" — scoped file/chat collections).
- **Access control (new, this fork):** `user_group_router` (Groups/Curators, \$5), `join_link_router` + `join_link_public_router` (\$5), `pat_router` (Personal Access Tokens).
- **Craft:** `build_router` / `build_admin_router` / `public_build_router` (\$8).
- **Tools/skills:** `tool_router` / `admin_tool_router`, `skill_router`, `input_prompt_router` / `admin_input_prompt_router`.
- **Web/knowledge search config:** `web_search_router` / `web_search_admin_router`.
- **Admin/config:** `admin_router` (misc admin ops), `settings_router` / `settings_admin_router`, `security_admin_router`, `llm_router` / `llm_admin_router`, `embedding_router` / `embedding_admin_router`, `search_settings_router`, `opensearch_migration_admin_router`, `tracing_admin_router`, `token_rate_limit_settings_router`, `kg_admin_router`, `code_interpreter_admin_router`.
- **Bots:** `slack_bot_management_router`, `discord_bot_router`.
- **Users/notifications/misc:** `user_router`, `notification_router`, `admin_banner_router`, `state_router` (health/version), `oauth_test_admin_router`.

EE-only routers layered by `ee/onyx/main.py`: `user_group_router` (the *other* `/manage/admin/user-group` implementation), `analytics_router`, `query_history_router`, `ee_query_router` / `search_router` (search-flow classification, search history), `standard_answer_router`, `ee_oauth_router`, `ee_document_cc_pair_router` (perm/group sync endpoints), `evals_router`, `enterprise_settings_router`, `token_rate_limit_settings_router` (EE variant), `usage_export_router`, and (multi-tenant only) `tenants_router`.

Error-handling convention

`onyx/error_handling/exceptions.py::OnyxError` is the **single exception type** business logic should raise (never raise raw `HTTPException` in business code, per the module's own docstring convention). It carries an `OnyxErrorCode` enum member (`onyx/error_handling/error_codes.py` — ~113 lines of codes grouped by HTTP status: `401` auth codes like `UNAUTHENTICATED` / `TOKEN_EXPIRED`, `403` authz codes like `INSUFFICIENT_PERMISSIONS` / `ADMIN_ONLY` / `EE_REQUIRED`, etc.) plus a human-readable `detail`. A global handler registered via `register_onyx_exception_handlers(application)` converts any uncaught `OnyxError` into a JSON body `{"error_code": "...", "detail": "..."} with the status code baked into the enum member (overridable per-raise for cases like proxying an upstream service's status). Generic RequestValidationError (422) and ValueError (400) also get dedicated JSON handlers so the API never leaks a raw stack-trace-shaped 500 for expected failure modes.`

11. Existing APIs Reference

All paths below are relative to the API root (no global prefix by default). CE routes are defined under `backend/onyx/server/`; EE routes under `backend/ee/onyx/server/` (mounted only when the EE app is running, which is always in this fork — see tier-gating note in §3).

Auth (fastapi-users, Basic only)

- `POST /auth/login`, `POST /auth/logout` — session login/logout.
- `POST /auth/register` — create account (subject to invite-list or Join Link, \$5).
- `POST /auth/forgot-password`, `POST /auth/reset-password` — password reset flow.
- `POST /auth/request-verify-token`, `POST /auth/verify` — email verification.
- `POST /auth/jwt/refresh` — session refresh.
- `GET/PATCH /users/me` — fastapi-users' own current-user endpoint (distinct from `onyx`'s richer `/me` below).
- `POST /auth/mobile/*` — mobile app session bootstrap.
- `POST /auth/captcha/oauth-verify` — captcha token verification gate.
- `POST /password/change-password`, `POST /password/reset_password` — Onyx's own password endpoints layered alongside fastapi-users'.

Users & permissions

- `GET /me` — current user's full profile. `GET /me/permissions` — effective permission set. `GET /get-user-role`.
- `PATCH /manage/set-user-role`, `PATCH /manage/admin/users/craft-enabled` — admin role/Craft-access management.
- `GET /manage/users`, `/manage/users/accepted`, `/manage/users/accepted/all`, `/manage/users/counts`, `/manage/users/invited`, `/manage/users/download` — user listing/export.
- `PUT /manage/admin/users` (invite), `PATCH /manage/admin/remove-invited-user`, `/manage/admin/deactivate-user`, `/manage/admin/activate-user`, `DELETE /manage/admin/delete-user`.
- `GET /manage/admin/valid-domains` — allowed signup email domains.
- `PATCH /temperature-override-enabled`, `/shortcut-enabled`, `/paste-as-tile`, `/auto-scroll`, `/user/theme-preference`, `/user/language`, `/user/chat-background`, `/user/default-app-mode`, `/user/default-model`, `/user/personalization`, `/user/pinned-assistants`, `/user/assistant-list/update/{id}`, `/user/assistant/{id}/preferences` — per-user preference writes.
- `GET /user/assistant/preferences`, `GET /user/files/recent`.
- `GET/POST/DELETE /user/pats` — Personal Access Tokens; `GET /user/pats/scopes`.

Groups / Curators / Join Links (new)

- `POST/GET /manage/admin/groups`, `PATCH /manage/admin/groups/rename`, `PATCH/DELETE /manage/admin/groups/{id}`, `POST /manage/admin/groups/{id}/add-users`, `PATCH /manage/admin/groups/{id}/agents` — basic group CRUD/membership/agent-sharing.
- `GET /manage/groups/minimal` — lightweight group list for pickers.
- `POST/GET /manage/admin/groups/{group_id}/join-links`, `DELETE /manage/admin/groups/{group_id}/join-links/{link_id}` — Join Link management (admin/curator).
- `GET /join-link/{token}`, `POST /join-link/{token}/redeem` — public Join Link lookup/redemption.
- EE variant: `GET/POST/PATCH/DELETE /manage/admin/user-group*`, `/manage/admin/user-group/{id}/permissions`, `/manage/admin/user-group/{id}/set-curator`, `/manage/admin/user-group/{id}/add-users`, `/manage/admin/user-group/{id}/agents`, `GET /user-groups/minimal` — the document-ACL-oriented group system.

Connectors, credentials, cc-pairs

- `GET/POST/PATCH/DELETE /manage/connector` (admin CRUD), `GET /manage/connector/status`, `/manage/connector/failed-indexing-status`, `POST /manage/connector/indexing-status`, `POST /manage/connector/run-once`, `POST /manage/connector/file/upload`, `GET/POST /manage/connector/{id}/files*`.
- `GET /connector`, `/connector/{id}`, `/indexed-sources`, `/connector-status`, `POST /connector-request`.
- OAuth for connector credential acquisition: `GET /connector/gmail|google-drive/authorize/{credential_id}` + `/callback`, `PUT /manage/connector/google-drive|gmail/service-account-credential`, `GET /manage/connector/google-drive/check-auth/{credential_id}`, `/connector/oauth/authorize/{source}`, `/connector/oauth/callback/{source}`, `/connector/oauth/details/{source}` (standard OAuth connectors), plus EE `ee/oauth/connector/{confluence|google-drive|slack}/callback` and `/prepare-authorization-request`.
- `GET/POST/PUT/PATCH/DELETE /manage/credential*`, `/credential`, `/credential/{id}`, `/credential/private-key`, `/credential/force/{id}`.

- `GET/PUT/DELETE /manage/cc-pair/{id}* — status, name, property, prune, errors, index-attempts, permission/external-group-sync-attempts (EE: /manage/cc-pair/{id}/sync-permissions, /sync-groups).`
- `POST /manage/indexing/targeted-reindex`, `GET /manage/indexing/targeted-reindex/{job_id}`.
- `GET /document-size-info`, `/chunk-info`.
- `GET/POST/DELETE /onyx-api/ingestion — external raw-document ingestion API; GET /onyx-api/connector-docs/{cc_pair_id}.`

Personas / Agents / Document Sets / Hierarchy / Projects / Tools / Skills

- `GET/POST/PATCH/DELETE /persona`, `/admin/persona/*` (create/list/update/share/transfer-ownership/labels/avatar/undelete/listed/public/featured/display-priorities).
- `GET/PATCH /admin/default-assistant/configuration`.
- `POST/PATCH/DELETE /manage/document-set`, `GET /document-set`, `/document-set-public`.
- `GET/POST /hierarchy` nodes/search/documents endpoints (constants-defined paths).
- `GET/POST/PATCH/DELETE /user/projects*` — Projects (file/chat scoping), token-count endpoints.
- `GET/PATCH /tool`, `/admin/tool`.
- `GET/POST/PATCH/DELETE /skills` (user_router).
- `GET/POST/PATCH/DELETE /input_prompt`, `/admin/input_prompt`.

Chat & Search

- `POST /chat/send-chat-message — the core streaming/non-streaming answer endpoint.`
- `GET /chat/get-user-chat-sessions`, `/chat/get-chat-session/{id}`, `POST /chat/create-chat-session`, `PUT /chat/rename-chat-session`, `/chat/update-chat-session-temperature`, `/chat/update-chat-session-model`, `PATCH /chat/chat-session/{id}`, `DELETE /chat/delete-chat-session/{id}`, `/chat/delete-all-chat-sessions`.
- `PUT /chat/set-message-as-latest`, `/chat/set-preferred-response`.
- `POST /chat/create-chat-message-feedback`, `DELETE /chat/remove-chat-message-feedback`.
- `GET /chat/max-selected-document-tokens`, `/chat/available-context-tokens/{id}`.
- `POST /chat/seed-chat-session-from-slack`, `GET /chat/file/{file_id}`, `GET /chat/search`, `GET /chat/chat-session/{id}/resume-stream`, `POST /chat/stop-chat-session/{id}`.
- `POST /query/search`, `GET /query/valid-tags`, `GET /admin/...` (admin query endpoints).
- `POST /search` (features/search — assistant-scoped one-shot search).
- EE: `GET /query/standard-answer`, `POST /search/search-flow-classification`, `GET /search/search-history`.

Web search & content providers

- `POST /web-search/search`, `/web-search/search-lite`, `/web-search/open-urls`.
- `GET/POST /admin/web-search/search-providers`, `.../activate`, `.../deactivate`, `POST .../test`, similarly for `/admin/web-search/content-providers` (+ `reset-default`).

LLM / Embedding / Search-settings admin

- `GET /admin/llm/custom-provider-names`, `/built-in/options[/provider]`, `POST /admin/llm/test[/default]`, `GET/PUT /admin/llm/provider`, `DELETE /admin/llm/provider/{id}`, `POST /admin/llm/default[-vision]`, `GET /admin/llm/auto-config`, `/vision-providers`, `/provider-contextual-cost`, `GET /llm/provider`, `/llm/persona/{id}/providers`, `POST /admin/llm/{bedrock|ollama|openrouter|lm-studio|litellm|bifrost|nebius-tokenfactory|openai-compatible}/available-models`.
- `POST /admin/embedding/test-embedding`, `GET/PUT/DELETE /admin/embedding[/embedding-provider]`.
- `POST /search-settings/set-new-search-settings`, `/cancel-new-embedding`, `DELETE /delete-search-settings`, `GET /get-current-search-settings`, `/get-secondary-search-settings`, `/get-all-search-settings`, `POST /update-inference-settings`, `GET/PUT/DELETE .../unstructured-api-key-set`.
- `GET /admin/opensearch-migration/status`, `/retrieval`, `PUT /admin/opensearch-migration/retrieval — controls the Vespa→OpenSearch cutover.`

Admin/settings/security/tracing

- GET/PUT /admin/settings, /settings; GET/PUT /admin/security.
- GET/POST/DELETE /admin/tracing/providers, POST /admin/tracing/providers/test, /providers/{type}/adopt-env.
- GET /admin/doc-boosts, POST /admin/doc-boosts, /admin/doc-hidden, GET /admin/genai-api-key/validate, POST /admin/deletion-attempt, GET /admin/indexing/failed-documents.
- GET /health, /auth/type, /version, /versions.
- GET/POST/DELETE /admin/token-rate-limits/global, PUT/DELETE /rate-limit/{id} (EE variant adds /user-groups, /user-group/{id}, /users).
- GET/PUT /admin/banner, DELETE /admin/banner.
- GET/PUT /admin/code-interpreter, /admin/code-interpreter/health.

Bots

- GET/POST/PATCH/DELETE /manage/admin/slack-app/channel[/]{id}, /manage/admin/slack-app/bots[/]{id}, /manage/admin/slack-app/bots/{id}/config.
- GET/POST/DELETE /manage/admin/discord-bot/config, /service-api-key, GET/POST /guilds, GET/PATCH/DELETE /guilds/{id}.

Craft (/build/*, /sessions/*, /approvals/*, /user-library/*, /scheduled-tasks/*)

- GET /build/base-instructions, /build/limit.
- GET/POST /sessions, GET /sessions/{id}, /sessions/{id}/sandbox-status, POST /sessions/{id}/generate-name, PUT /sessions/{id}/name, PATCH /sessions/{id}/public, DELETE /sessions/{id}, POST /sessions/{id}/restore, /sessions/{id}/snapshot, /sessions/{id}/opencode-history-snapshot.
- GET /sessions/{id}/files, /sessions/{id}/artifacts/{path}, /sessions/{id}/export-docx/{path}, /sessions/{id}/pptx-preview/{path}, /sessions/{id}/webapp-info, /sessions/{id}/webapp-download, /sessions/{id}/download-directory/{path}, POST /sessions/{id}/upload, DELETE /sessions/{id}/files/{path}.
- GET /sessions/{id}/messages, POST /sessions/{id}/send-message, /sessions/{id}/interrupt.
- GET /sessions/{id}/turns/active, /sessions/{id}/turns/{turn_id}/events.
- GET /approvals/sessions/{id}/live, POST /approvals/{id}/decision, /approvals/{id}/session-grant.
- GET/POST /scheduled-tasks, GET/PATCH/DELETE /scheduled-tasks/{id}, POST /scheduled-tasks/{id}/run-now, GET /scheduled-tasks/{id}/runs.
- GET /user-library/tree, POST /user-library/upload[-zip], /directories, DELETE /user-library/files/{id}.
- GET/POST/PATCH/DELETE /build/apps* (External Apps: built-in/custom, credentials, OAuth start/callback, connect decision).

Federated search & Knowledge Graph

- POST /federated, GET /federated/{id}/entities, ... /credentials/schema, /federated/sources/{source}/configuration/schema, ... /credentials/schema, POST ... /credentials/validate, GET /federated/{id}/authorize, POST /federated/callback, GET /federated, /oauth-status, GET/PUT/DELETE /federated/{id}, DELETE /federated/{id}/oauth.
- GET /admin/kg/exposed, PUT /admin/kg/reset, GET/PUT /admin/kg/config, GET/PUT /admin/kg/entity-types.

EE-only: Analytics, Query History, Reporting, License, Evals, Standard Answers

- GET /analytics/admin/{query|user|onyxbot}, /analytics/admin/persona/messages, /persona/unique-users, GET /analytics/assistant/{id}/stats.
- GET /admin/chat-sessions, /admin/chat-session-history[/]{id}, /admin/query-history/list, POST /admin/query-history/start-export, GET /admin/query-history/export-status, /download.
- POST /admin/usage-report, GET /admin/usage-report[/]{name}.
- GET /license, /license/seats, POST /license/claim, /license/upload, /license/refresh, DELETE /license.
- POST /evals/eval_run.
- GET/POST/PATCH/DELETE /manage/admin/standard-answer[/]{id}, /manage/admin/standard-answer/category[/]{id}.
- GET /enterprise-settings (basic_router).

- (Present but unrouted/dead: `ee/onyx/server/billing/*`, `ee/onyx/server/tenants/billing_api.py`, `.../proxy.py` — Stripe checkout/portal/seats endpoints exist in source but are never mounted by `ee/onyx/main.py`.)
- Multi-tenant only (`MULTI_TENANT=true`): `POST /tenants/impersonate`, `GET/POST /tenants/anonymous-user-path`, `POST /tenants/anonymous-user`, `POST /tenants/leave-team`, `GET /tenants/existing-team-by-domain`, `POST /tenants/users/invite/*`, `GET /tenants/users/pending`.

12. Observability / Tracing

`onyx/tracing/` provides a pluggable LLM-call tracing layer independent of Sentry (which handles generic exception monitoring). Two concrete backends exist and are selected at runtime via configured `TracingProviderConfig` rows: **Braintrust** (`braintrust_tracing_processor.py`) and **Langfuse** (`langfuse_tracing_processor.py`), fronted by a `dynamic_processor.py` dispatcher and initialized by `setup.py::setup_tracing()` at app/worker startup.

The `LLMFlow` tagging convention

`onyx/tracing/flows.py::LLMFlow` (a `StrEnum`) is the single source of truth for what a given LLM-generation span *represents* — deliberately **not** which provider/model served it (that's captured separately on the span's `model_config`). Every LLM call site is expected to pass an explicit flow tag when opening a span, grouped by concern:

- **Chat/agent:** `CHAT_RESPONSE`, `CHAT_HISTORY_SUMMARIZATION`.
- **Secondary LLM flows** (small, cheap calls that support the main chat flow): `SEMANTIC_QUERY_REPHRASE`, `KEYWORD_QUERY_EXPANSION`, `SOURCE_FILTER_EXTRACTION`, `TIME_FILTER_EXTRACTION`, `CLASSIFY_SECTION_RELEVANCE`, `SELECT_SECTIONS_FOR_EXPANSION`, `CHAT_SESSION_NAMING`, `MEMORY_UPDATE`.
- **Craft:** `BUILD_SESSION_NAMING`.
- **Federated search helpers:** `SLACK_DATE_EXTRACTION`, `SLACK_QUERY_EXPANSION`.
- **Indexing** (runs inside the docprocessing worker): `CONTEXTUAL_RAG_DOC_SUMMARY`, `CONTEXTUAL_RAG_CHUNK_CONTEXT`.
- **Knowledge graph:** `KG_DOCUMENT_CLASSIFICATION`, `KG_DEEP_EXTRACTION`.
- **Embeddings/rerank/intent** (cross-process calls into the model server): `EMBED_QUERY`, `EMBED_PASSAGE`, `RERANK`, `INTENT_CLASSIFICATION`.
- **Sentinels:** `UNTAGGED_INVOKE` / `UNTAGGED_STREAM` — emitted automatically by the `LLM` base class's auto-wrap fallback (§7) whenever a call site didn't open an explicit tagged span. Seeing these in a dashboard is the signal to go add a proper `llm_generation_span( ... )` call at that call site — the enum module comment explicitly frames this as an instrumentation-coverage mechanism, not an accepted steady-state tag.
- A couple of enum members (`IMAGE_GENERATION`, `IMAGE_EDIT`, `STT`, `TTS`) are **vestigial** — they correspond to Image Generation and Voice features that this fork has removed; the tags remain in the enum (likely because removing them would break historical trace data / dashboards that reference them) but there is no live code path that ever emits them.

Because every `LLM` subclass has `invoke` / `stream` auto-wrapped via `__init_subclass__` (see §7), the guarantee is: **every LLM call anywhere in the codebase produces a trace span**, whether or not the call site remembered to tag it — untagged ones just show up as `untagged_invoke` / `untagged_stream` instead of a meaningful flow name, which is a much easier gap to spot and fix than a silently-missing span.